

Faculty of Science and Information Technology
Department of Computer Science and Engineering

Cubicle

A Self-Hosted Serverless Functions Platform

A project report submitted for **CSE315 | Software Engineering**

SUBMITTED BY

Name	Student ID	Email
Tamzidul Haque	241-15-877	tamzidul241-15-877@diu.edu.bd
Umma Iman Monea	241-15-540	iman241-15-540@diu.edu.bd
Puja Chowdhury	241-15-493	puja241-15-493@diu.edu.bd
Tamima Rahman	241-15-030	rahman241-15-030@diu.edu.bd
Shagato Chowdhury	241-15-331	shagato241-15-331@diu.edu.bd

SUPERVISED BY

Dr. Abdus Sattar
Associate Professor and Director, M.Sc. in CSE Department of Computer Science and Engineering Faculty of Science and Information Technology, Daffodil International University

9 August 2026

Table of Contents

Table of Contents . . . . . **i**

List of Figures and Tables . . . . . **ii**

Project Story . . . . . **iii**

1 Introduction **1**

1.1 Introduction 1

1.2 Motivation 1

1.3 Objectives 2

1.4 Related Works 2

2 Software Requirements Specifications **7**

2.1 Model Selection 7

2.2 Functional and Non-Functional Requirements 9

2.3 Use Case Diagram 10

2.4 Use Case Description 10

2.5 Activity Diagram 16

2.6 Sequence Diagram 16

2.7 Class Diagram 20

3 System Design and Implementation **26**

3.1 System Architecture 26

3.2 Graphical User Interface 31

4 Project Planning and Requirements **36**

4.1 Project Estimation and Planning 36

4.2 Hardware and Software Requirements 39

5 Management and Considerations **40**

5.1 Ethical, Legal and Security Issues 40

5.2 Limitations and Challenges 41

5.3 Team Collaboration and Time Management 43

Acknowledgement **45**

References **45**

List of Figures and Tables

List of Figures

1.1 Existing platforms: OpenWhisk and OpenFaaS 4
1.2 Existing platforms: Knative and Fission 5
1.3 The hosted reference point, and Cubicle 6

2.1 The incremental process model as applied 8
2.2 Use case diagram 11
2.3 Activity diagram: deploying a function version 17
2.4 Activity diagram: admission control 18
2.5 Activity diagram: scheduling and execution 19
2.6 Class diagram: the control plane runtime 21
2.7 Sequence diagram: first-run setup 22
2.8 Sequence diagram: deploying a version 23
2.9 Sequence diagram: invoking a function on a cold start 24
2.10 Class diagram: the persistent domain model 25

3.1 Layered system architecture 27
3.2 Deployment view 29
3.3 State machine: the life of an isolate 30
3.4 The console: overview and playground 32
3.5 The console: metering and logs 33
3.6 The console: database manager and settings 34
3.7 Navigation map of the console 35

4.1 Project schedule 38

5.1 Trust boundaries and the envelope encryption path 42

List of Tables

1.1 Project objectives and where each is demonstrated 2
1.2 Comparison with existing open source function platforms 2

2.1 Process model selection 7
2.2 Functional requirements 9
2.3 Non-functional requirements 10

4.1 Size of the delivered system 36
4.2 Unadjusted function point count 36
4.3 Risk register, with the mitigation actually applied 39
4.4 Server requirements 39

5.1 Principal third party components and their licences 40
5.2 Known limitations 42
5.3 Division of work within the group 43

Project Story

Localoy operates a website called `localoy.app`, through which customers find local businesses, browse what those businesses offer, and complete a booking and a payment without leaving the site.

The company began with a single team working on a single codebase, but the platform is now built as a set of microservices. Each service owns its own database, is deployed independently, and has a team responsible for it, covering listings, booking, payments, partner integrations and customer accounts. Every server belongs to the company, and the company intends to keep it that way.

Releasing to production is deliberately slow. Every change is reviewed by another engineer, tested by the full automated suite, and left running on staging for several days before it ships in a scheduled window with a named engineer on call. The company adopted this process after a difficult year, and the year that followed was measurably better, so nobody wishes to see it weakened.

The difficulty is that the process costs the same regardless of the size of the change, while an increasing share of the incoming work is very small.

When an engineer is asked to add a small piece of server-side behaviour, only two options are available and neither is satisfactory. The first is to place the new code inside an existing service, where it inherits that service's release process and database even though it has nothing to do with the purpose of that service, so that a form storing five fields waits in the same queue as a change to checkout. The second is to create a service of its own, which requires a repository, a deployment pipeline, a database, monitoring and an engineer on call at three in the morning. That cost is reasonable for a genuine business capability, but it is plainly disproportionate for a feedback form.

The consequence is more serious than delay, because work of this size is frequently never delivered at all.

A survey to be sent after a completed booking was requested on the eleventh of February and estimated at half a day of effort; it has since appeared on three release plans and has still not been built. The marketing team, who wanted a feedback widget on three landing pages, eventually abandoned the request and adopted an external form product, with the result that several months of customer responses now reside in an account that nobody administers. A callback written for a partner amounted to fewer than a hundred lines of code and took nine days to deliver, which the partner remarked upon. A small tool for correcting wrongly tagged listings was estimated and never started, so a colleague continues to correct those records by hand each week. The single request that was delivered, a waiting list for a beta feature, consumed an entire release slot and delayed that feature by a full cycle. Four further requests are queued, and most are expected to be dropped without discussion.

An expedited release path was proposed internally and rejected for a sound reason: however quickly such a change is shipped, it still executes inside a production service and still writes to a production database.

A second and less visible problem has developed alongside the first, in that certain small units of work are required by several services and owned by none of them. Photographs uploaded by businesses must be compressed before they are stored, and their dominant colours must then be extracted so that listing pages can be tinted to match. Each service either implements this itself or depends on a shared library, and neither approach is satisfactory. Independent implementations diverge over time, so the same photograph is stored at different sizes depending on which part of the platform received it, while a shared library resolves that inconsistency and introduces a worse one, because altering a single compression setting obliges every consuming service to adopt the new version and pass through the full release process.

Localoy therefore wishes these units of work to be hosted separately and invoked over HTTP by the production services. The company accepts that this places them closer to the live request path than a feedback form would be, and requires in exchange that a calling service degrade gracefully rather than fail when the platform cannot be reached.

The available commercial and open source options were examined and set aside. A managed provider would introduce an external supplier, recurring invoices and a bill that may rise unpredictably, and it would place customer feedback on hardware owned by another company. A hosted survey product addresses only a subset of the requests, and the abandoned marketing account demonstrates where that approach leads. Building a dedicated service for each case remains appropriate for a genuine capability, but in these cases the surrounding infrastructure would cost considerably more than the code it contains. The established self-hosted platforms came closest to meeting the need and were nevertheless rejected for a practical reason, since each of them assumes the presence of an engineer whose responsibility is to operate it, and the infrastructure group has nobody available.

Localoy requires a system that is purpose-built, delivered to the company, and thereafter operated by the company itself. It must not form part of any production service, nor be deployed alongside one, and it must not become a further service that requires monitoring and out-of-hours attention. It must run on company hardware with administrative access held by the company, and it must not introduce a recurring charge in the form of a subscription, a usage fee or a licence for each seat. No customer data may leave the company’s infrastructure, and no component may report information outward. The source code must belong to Localoy outright, and the system must be operable by the staff already employed, without recruitment and without requiring anyone to learn an unfamiliar discipline.

The resources available are equally well defined, and the company would prefer to learn immediately, rather than after six months, that they are insufficient. One virtual server can be allocated, providing four processor cores, eight gigabytes of memory and thirty gigabytes of storage, and a second identical machine can be provided if capacity demands it. A subdomain of `localoy.app` will be made available, and an infrastructure engineer can devote half a day each week to the system and no more than that.

The anticipated load consists of fifteen to forty functions, most of which will be idle for the greater part of the day. The heaviest case is the post-booking survey, which is expected to receive approximately two thousand submissions over forty-eight hours, and the company requires headroom for fifty requests per second. Payloads will be small and predominantly textual, with the exception of the images. The daily users will be product engineers within the service teams rather than infrastructure specialists, so an engineer encountering the system for the first time must be able to use it with nothing but a browser and the documentation.

Several matters are excluded from the request explicitly. The production services are not to be modified, and a function requiring data from one of them will obtain it through that service’s ordinary interface. No existing functionality is to be migrated out of those services. Neither a form nor a survey front end is required, and multi-region deployment and automatic failover are likewise unnecessary. Support for a single programming language is sufficient at the outset. Finally, Localoy accepts that the platform may occasionally be unavailable, provided that the services calling it degrade rather than fail.

Cubicle is the proposal submitted in response to this request, and the remainder of this report documents it.

CHAPTER 1

Introduction

1.1 Introduction

Serverless computing, or Function as a Service, is a way of running code in which the unit of deployment is a single function rather than a server. The developer writes a handler and uploads it; the platform allocates a machine, routes the request, scales the number of workers with demand and reclaims the resources when traffic stops, and nothing is charged while nothing is running [9]. AWS Lambda [6] and its equivalents serve a large share of event driven workloads on that model. The cost of the convenience is not only monetary: code runs on hardware the developer does not control, in a jurisdiction they may not have chosen, on a platform whose interface and pricing can change, and for a class of users and a class of data that trade is not acceptable.

Cubicle makes the opposite trade. It gives the developer experience of a hosted platform, namely write a handler, press deploy, receive a URL, on a machine the operator owns, with no account to create anywhere, no telemetry, and no component that contacts a service outside the machine unless the operator configures one. Installation is a single command, and pointing a domain name at the machine is enough to obtain and renew a TLS certificate.

Concretely, Cubicle is a control plane written in Python with FastAPI [2] and SQLAlchemy [3], a browser console in React [4] and TypeScript, a set of container images that each carry one language runtime, and a command line client. It uses the Docker Engine API [1] as its scheduling substrate: a deployed function becomes an immutable version in a container volume, and a request is served by a container, called an *isolate*, that mounts that volume read only and stays warm between requests. PostgreSQL holds durable state, Redis holds sessions and the shared runtime context, and Caddy [5] terminates TLS at the edge.

The delivered system is 36,300 lines across five components, exposes 110 API endpoints, supports seven runtimes across Python and Node.js, is covered by 127 automated tests, and is distributed as a Docker Compose project with one install script under the Apache License 2.0.

1.2 Motivation

Four observations led to this project.

The convenience of serverless is separable from the cloud. The model is a scheduler, a router, an isolation mechanism and a metering system, and each has a mature open implementation. What the commercial platforms sell is the integration of those pieces, which is a software engineering problem rather than a capital one, so it can be solved for a single machine.

Data residency is a requirement, not a preference. Data protection regulation places obligations on where personal data is processed and who may access it. The most direct answer to the question of whose employees could in principle read a record is that the process ran on a machine in the organisation's own building, and a platform that requires an account somewhere cannot give it.

The cost curve of hosted functions turns predictably. Hosted pricing is close to free at low volume and considerably less so at sustained volume, because it is charged per gigabyte second of allocated memory and per million requests. A small server can absorb a workload whose hosted bill is a multiple of its monthly cost, provided something schedules the work on it, and Cubicle records enough per invocation to display that comparison from its own records.

Existing self-hosted options assume Kubernetes. The mature open platforms are almost all Kubernetes controllers, which is an excellent answer for an organisation that already runs it and a second system to learn, staff and patch for one that does not. The gap between one Linux server with Docker on it and a Kubernetes cluster is where a large number of small teams, research groups and university departments live, and Cubicle targets it deliberately.

1.3 Objectives

Table 1.1: Project objectives and where each is demonstrated

No.	Objective	Demonstrated in
O1	Build a platform that installs on a single Linux host with one command and requires no account with any external service.	3.1
O2	Provide immutable versioned deployment, so a failed build cannot affect the version currently serving traffic.	2.6, 3.1.4
O3	Keep containers warm between requests and reclaim them when idle, so an unused function costs nothing and a busy one pays no start-up cost.	3.1.4
O4	Support several languages through a runtime registry, so adding one is a configuration change rather than a scheduler change.	3.1.4
O5	Isolate tenants at two levels: a cluster boundary that nothing crosses, and a per account grant deciding which clusters an account may address.	3.1, 5.1.3
O6	Enforce resource ceilings per cluster that account for every consumer, including data services and stored volumes.	3.1.4
O7	Encrypt every stored secret so that a copy of the database alone recovers nothing.	5.1.3
O8	Record every invocation and derive all metrics, metering and chargeback from those records rather than from sampling.	3.1.4
O9	Provide a console covering the whole lifecycle: authoring, deployment, configuration, data services, logs, scheduling and updates.	3.2
O10	Run functions on a recurring schedule with exactly once semantics.	3.1.4

1.4 Related Works

Baldini and colleagues [9] survey the model and name the open problems that still shape platform design: cold start latency, the difficulty of composing functions, and the absence of a useful state abstraction between invocations. Cubicle answers the last in a limited way through a *session context*, a small key value document held in Redis, scoped to a namespace and keyed by a caller-supplied header, which the functions of that namespace read and write according to a per function declaration.

Platforms differ mainly in what they use as the isolation boundary. Micro virtual machines give a hardware enforced boundary and start in tens of milliseconds but need virtualisation support; language level isolates make cold start almost free but restrict the platform to one language family. Fission, another Kubernetes controller, uses a pre-warmed pool that is the same idea as the isolate pool here. Cubicle uses OS containers, because they are the only mechanism available on an unmodified Docker host; the containment is hardened as far as that allows, and the residual risk is stated in Section 5.1.3.

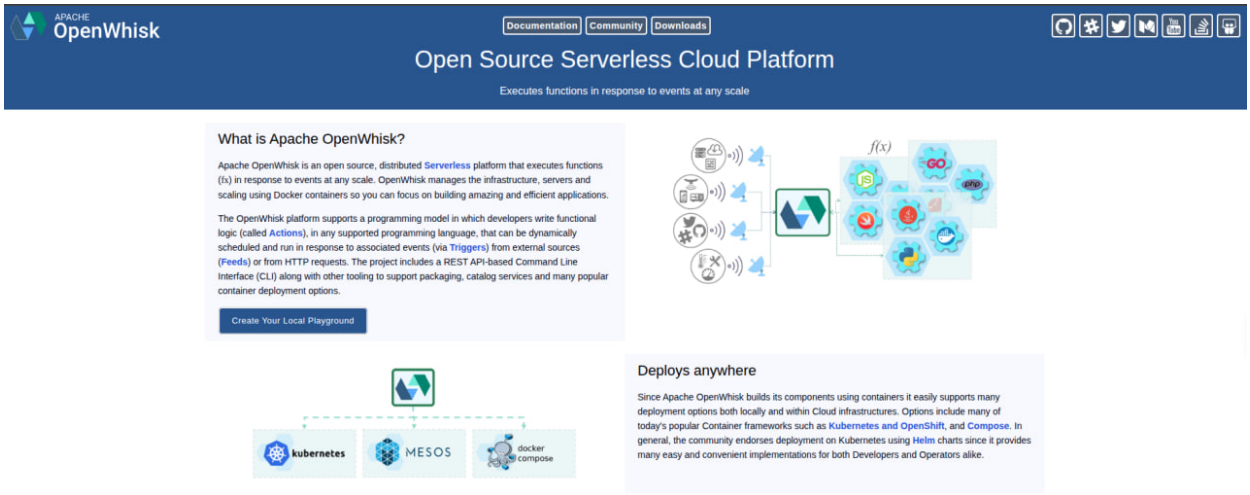
Figures 1.1 to 1.3 show each of these platforms as its own interface presents it, and Table 1.2 compares them on the properties that matter to the operator of a single server.

Table 1.2: Comparison with existing open source function platforms

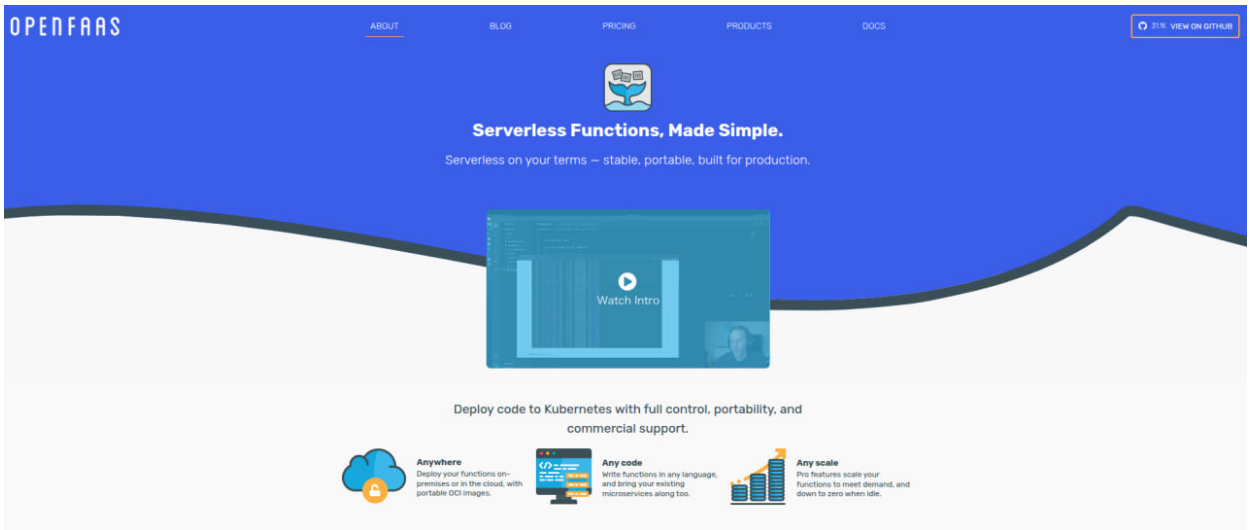
System	Orchestrator	Position	Console
Apache OpenWhisk [8]	Kubernetes, or Kafka and CouchDB	The most complete open implementation, and the heaviest to operate: several stateful services are required before the first function runs.	Third party
OpenFaaS [7]	Kubernetes, or Swarm in `faasd`	Closest in spirit, and `faasd` runs on one host. Functions are container images, so the developer supplies a Dockerfile and a build pipeline.	Yes
Knative	Kubernetes	A serving and eventing layer rather than a function platform; the developer brings an image and a service mesh.	No
Fission	Kubernetes	A pre-warmed pool removes most of the cold start, which is the same idea as the isolate pool here, but Kubernetes remains mandatory.	Yes
Cubicle	**Docker Engine API**	**One host, one command, no orchestrator. The developer supplies a handler file and a dependency list, never an image. Clusters, ceilings, data services, metering and a marketplace are part of the product.**	**Yes**

Four things are specific to this work: **no image build in the developer's workflow**, since the deployable artefact is source code, so a deploy takes seconds and no registry exists in the system; **clusters as a first**

class boundary on one instance; **resource ceilings that count everything**, including the memory held by a cluster's own databases and the size of its volumes; and **operability as a feature**, meaning reconciliation, a one press update, schedule triggers, a marketplace and a database manager. Cubicle does not claim a hard multi-tenant security boundary; that requires hardware virtualisation, and the consequence is stated in Section 5.1.3.

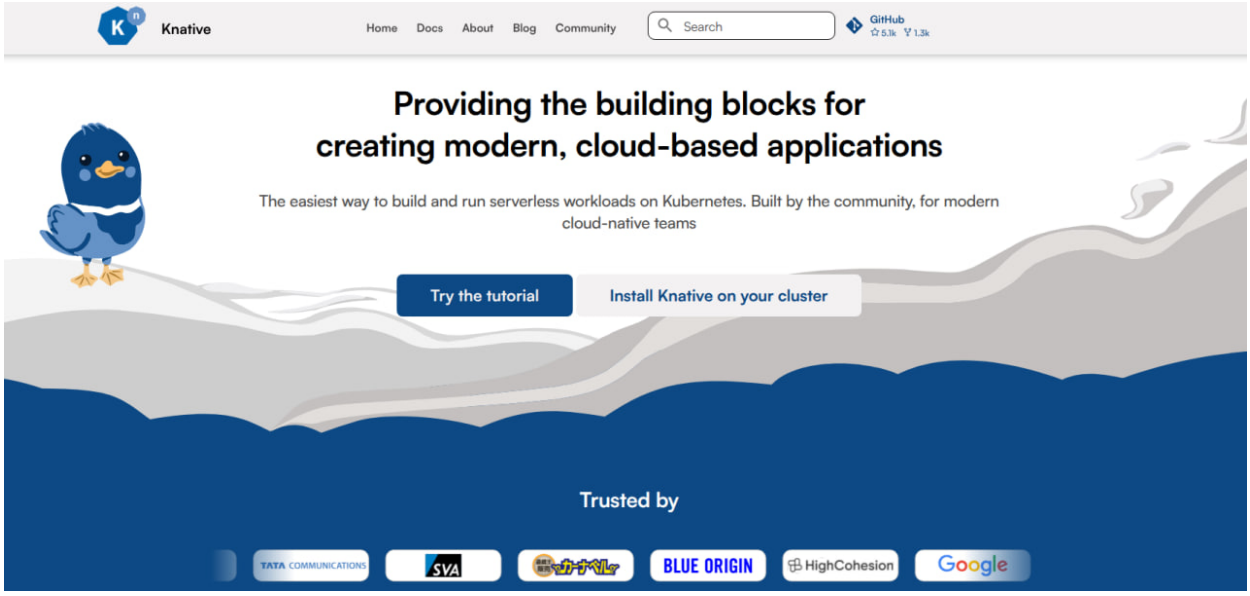


(a) Apache OpenWhisk. The most complete open implementation of the model, and the one that asks the most before it will run: a message bus, a document store and a controller must all be standing before the first action is invoked. The capability is real, and so is the operating burden it assumes somebody is there to carry.

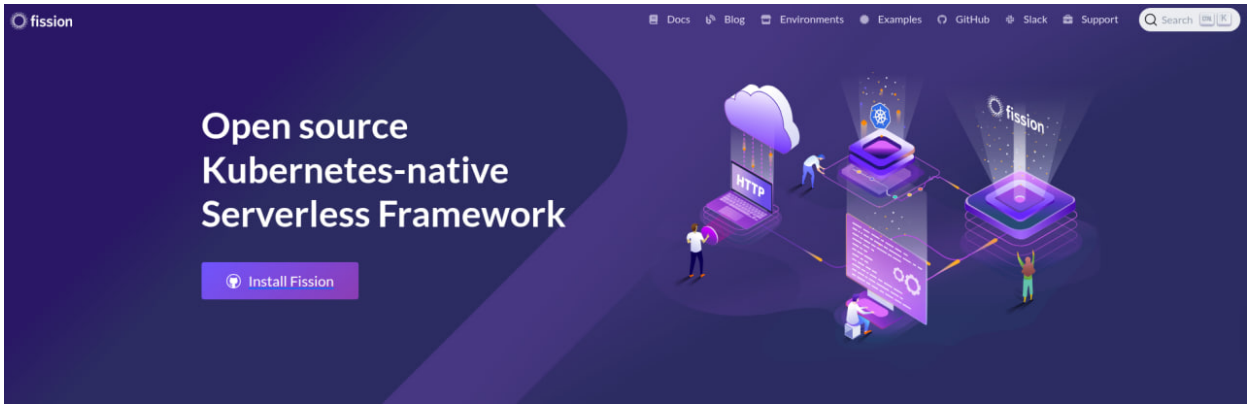


(b) OpenFaaS. The closest neighbour to this project, and the only one that runs on a single host without an orchestrator. The difference is what the developer hands over: OpenFaaS takes a container image, so a Dockerfile, a build pipeline and a registry sit between writing a function and running it.

Figure 1.1: The two most complete open implementations of the model. Both are capable and both presume an operator: one in the services it must be given, the other in the build chain it puts in front of the developer.

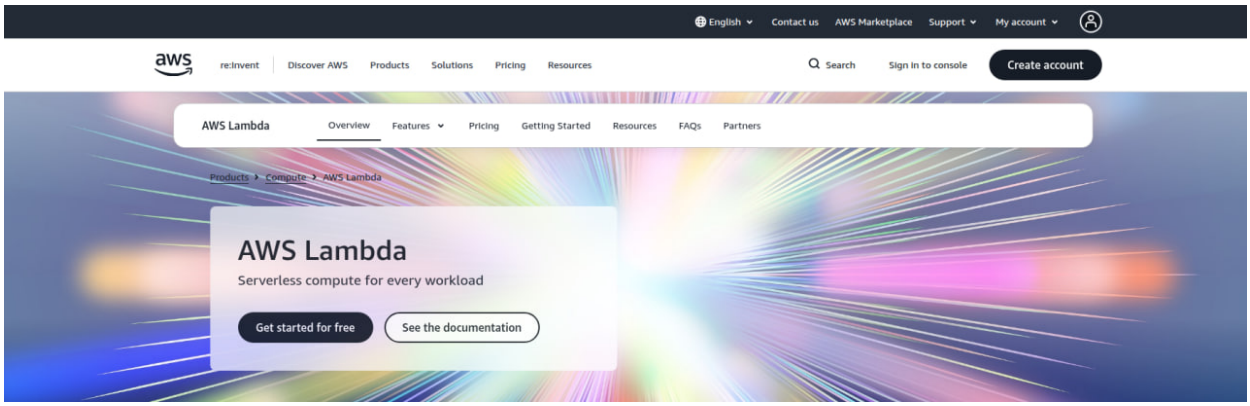


(a) Knative. A serving and eventing layer rather than a function platform. It scales a container to zero and back competently, but it starts from an image and a service mesh, and it ships no console at all, so everything an operator does here is done through Kubernetes itself.

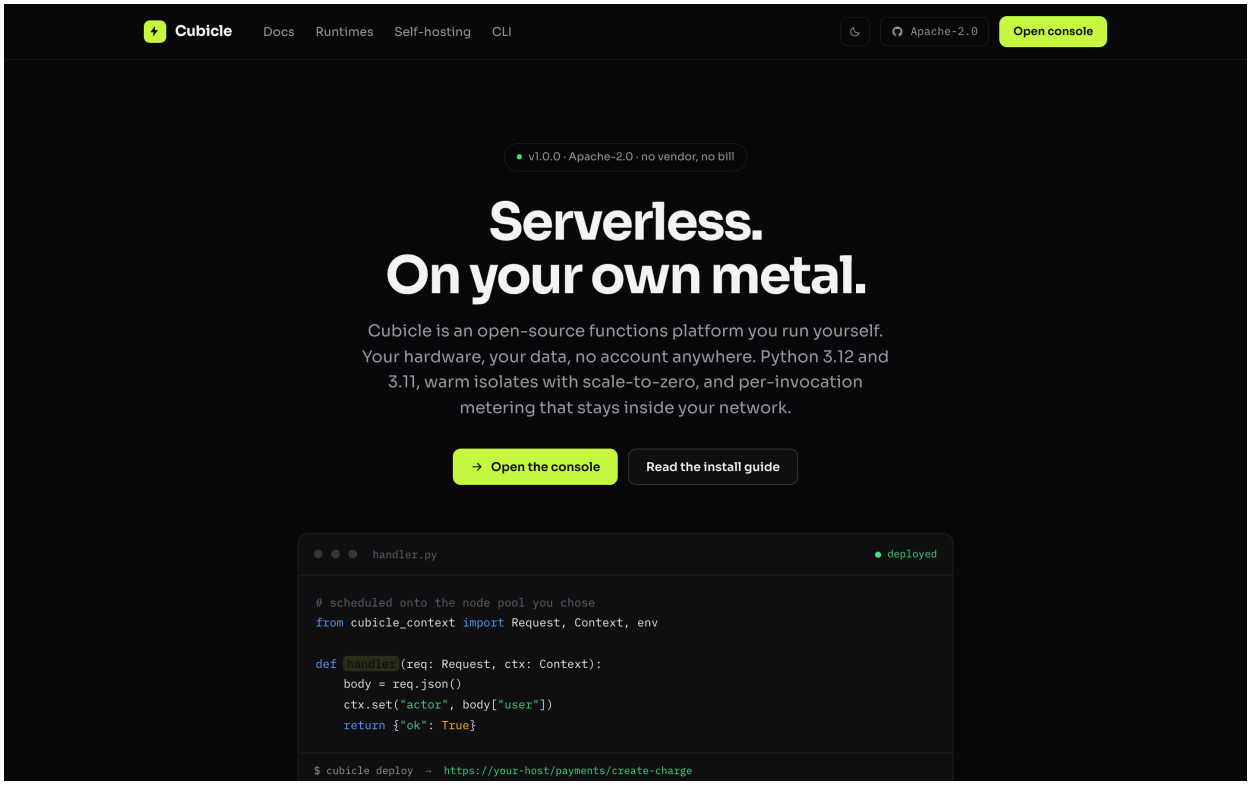


(b) Fission. Notable for keeping containers pre-warmed so a request rarely pays a cold start, which is the same insight as the isolate pool described in Section 3.1.4. The mechanism is right; the prerequisite is a Kubernetes cluster that somebody has to run.

Figure 1.2: Two Kubernetes-native approaches. Between them they demonstrate that the hard parts of the model are solved, and that the solutions are only available to an organisation already operating Kubernetes.



(a) AWS Lambda. The hosted service that defines what users now expect: no server to think about, a function that scales itself, and a bill only while it runs. It is also the trade this project refuses, since the code executes on hardware the developer neither owns nor can inspect, under pricing and interfaces that can change without them.



(b) Cubicle. The same promise, made on the operator's own hardware. One command installs the whole platform, there is no account to create anywhere, and nothing in the product contacts a service outside the machine. The console that follows this front page is shown in Section 3.2.

Figure 1.3: The hosted platform the model is defined by, and the system this report describes. The functionality on offer is deliberately similar; what differs is who owns the machine, who holds the data, and what the arrangement costs to keep.

CHAPTER 2

Software Requirements Specifications

2.1 Model Selection

A process model is a claim about how much is known at the start and how expensive a late change will be. Two facts characterise this project: the problem domain was well understood at the outset, because the commercial platforms define what a function platform must do, while the internal design was not, because container scheduling, warm pooling and reclamation policy could only be settled by measurement on a running system.

Table 2.1: Process model selection

Model	Assessment for this project	Verdict
Waterfall	Requires the design to be fixed before construction. The reclamation policy, the minimum memory a runtime serves at and the concurrency ceiling were all decided by experiment, so a frozen design would have been wrong when it was signed.	Rejected
Prototyping	Suits a project whose requirements are unclear. Here the requirements were clear and the risk lay in construction, so a throwaway prototype would have discarded the code that answered the open questions.	Rejected
Spiral	Its strength is formal risk analysis each cycle, which is proportionate to a large team carrying commercial risk, not to a five person student group.	Rejected
Scrum	The short cycle and the working system as the measure of progress were adopted. The full ceremony set was not, because the group met daily in person.	Partly adopted
Incremental	Each increment delivers a working, deployable system that is a superset of the last. The architecture is settled once, at the start, and every increment extends rather than replaces it. Risk is retired by having something running from the end of increment one.	**Selected**

The project used an **incremental model** with a single architecture phase at the front, shown in Figure 2.1. Three properties of the problem made this the right choice. The architecture is load bearing and had to come first, because the decision to make the deployable artefact source code rather than an image, and the decision to make a cluster the isolation boundary, reach into every layer. Each increment is independently useful, since after the first the system already ran functions. And measurement was part of construction: the 32 MB floor at which a Python isolate serves came from probing a running isolate, not from a specification.

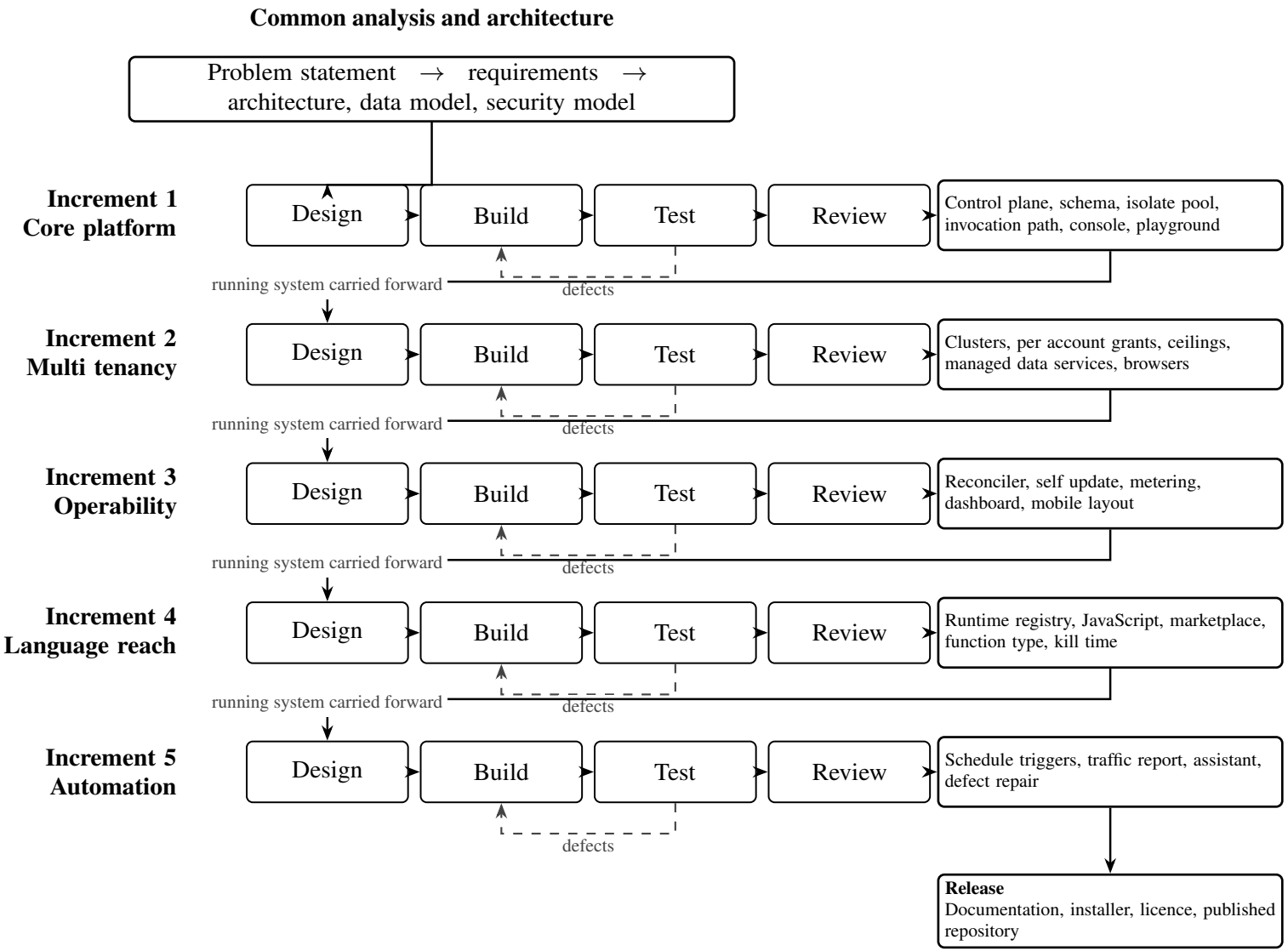


Figure 2.1: The incremental process model as applied. Analysis and architecture were done once; each increment then added a vertical slice through every layer and left a system that could be installed and used.

The repository records 63 commits across five increments, each of which left the system installable: the core platform, multi-tenancy, operability, language reach, and automation with defect repair. Section 4.1 maps them onto the schedule.

2.2 Functional and Non-Functional Requirements

Requirements carry a priority on the MoSCoW scale. Every requirement listed is implemented, and is exercised either by an automated test or by the console screen named in Section 3.2.

Table 2.2: Functional requirements

ID	Requirement	Priority
FR-01	Install from a single command, generating every secret, and obtain a TLS certificate automatically when a domain is supplied.	Must
FR-02	Create the only administrator account through a first-run wizard, with no registration step and no external service, reusing an existing configuration when re-run and never rotating a secret.	Must
FR-03	Support several clusters on one instance, each owning its namespaces, configuration, nodes, data services and metrics, with no data crossing the boundary.	Must
FR-04	Require every endpoint to name its cluster, by a mapped hostname or by the cluster slug in the path.	Must
FR-05	Restrict an account to its granted clusters, answer for an ungranted cluster exactly as for one that does not exist, and guarantee that the setup account always retains access.	Must
FR-06	Support the roles readonly, developer, admin and owner in increasing order of authority, and let a developer create a namespace and, within it, a function whose name is unique to that namespace, at the smallest resource configuration each control offers.	Must
FR-07	Accept a source bundle of an entry file and an optional dependency declaration, and reject anything else.	Must
FR-08	Produce an immutable numbered version on deploy, install its declared dependencies into private storage, and allow an earlier version to be restored.	Must
FR-09	Leave the version currently serving completely unaffected by a failed build, and retain the build log.	Must
FR-10	Serve an invocation from a container dedicated to one function version, mounted read only, kept warm between invocations and reclaimed after a per function idle period.	Must
FR-11	Bound concurrency by a per function maximum and an instance-wide ceiling, queueing beyond it rather than starting another container.	Must
FR-12	Return the request identifier, duration and cold start flag on every response, carry non-text bodies through byte for byte, and terminate a handler that exceeds its timeout with 504.	Must
FR-13	Hold cluster-wide configuration resolved at invocation time so a change needs no redeploy, storing secret values encrypted and never returning them in plaintext.	Must
FR-14	Share a session context between the functions of one namespace, keyed by a caller-supplied header, with per function read and write permission.	Should
FR-15	Provision a PostgreSQL and a Redis instance per cluster on request, wire them into every function without copying credentials, and browse and edit them from the console.	Should
FR-16	Support several language runtimes through a registry, shipping Python and JavaScript, installing the rest on demand, and allowing an installed image to be rebuilt from the console.	Should
FR-17	Browse a community registry, display a package in full before anything is created, and install it into a namespace without writing secrets on the user’s behalf.	Could
FR-18	Run an independent function on a cron schedule with a named timezone, and fire a due schedule exactly once even if two control plane processes evaluate it together.	Must
FR-19	Let a super administrator set a memory, processor and storage ceiling per cluster that no lesser role can raise, counting every consumer against it and refusing an excess allocation at admission.	Must
FR-20	Display the remaining capacity of the selected cluster continuously, recording every invocation and deriving every metric, meter and chargeback figure from those records rather than from sampling.	Must
FR-21	Stream handler output and control plane events live, reporting why a failed invocation failed rather than only its status code, and expose metrics in Prometheus format.	Must
FR-22	Compare recorded state against the engine’s actual state, present the differences and repair them on request.	Should
FR-23	Detect that the upstream branch has moved on and apply the update from the console in one action.	Could

ID	Requirement	Priority
FR-24	Expose a documented HTTP API covering every console capability, and a command line client with no dependency outside the standard library.	Must

Table 2.3: Non-functional requirements

ID	Characteristic	Requirement and acceptance criterion
NFR-01	Performance	Platform overhead on a warm invocation shall be in the low single digit milliseconds; a cold start shall complete within a few hundred milliseconds on a two core machine and be reported in a response header. No process shall be spawned per invocation.
NFR-02	Resource efficiency	A Python isolate shall serve at a 32 MB limit and a JavaScript isolate at 64 MB, these being the measured floors; an idle function shall hold no memory once reclaimed.
NFR-03	Reliability	The control plane shall be restartable at any moment, losing nothing but warm containers, and shall re-attach to containers whose version is still current.
NFR-04	Reliability	A build failure, a handler exception, a timeout and an unreachable engine shall each produce a distinct documented outcome rather than a generic failure.
NFR-05	Security	Passwords shall be stored only as argon2id hashes; secrets shall be encrypted with a per record AES-256-GCM key wrapped by a key derived from a root key held outside the database.
NFR-06	Security	Containers shall run unprivileged with a read only root filesystem, all capabilities dropped, no-new-privileges set, a process limit and capped memory and swap.
NFR-07	Security	Sessions shall be opaque server-side tokens so they can be revoked immediately; API keys shall be stored as a keyed hash and displayed once.
NFR-08	Privacy	No component shall contact any third party service, and the console shall load no asset from an external origin.
NFR-09	Usability	Installation to first deployed function shall need no configuration file and no Dockerfile; the console shall remain usable on a phone; a settings form shall apply only when saved.
NFR-10	Maintainability and portability	The scheduler shall know nothing about any specific language; schema changes shall be reversible migrations applied under a lock; the system shall run from one Compose project on x86-64 and ARM64.

The project works under three constraints. The deployment target is a single Linux host with a container engine, and Kubernetes is explicitly not a prerequisite. The isolation mechanism is the operating system container, because hardware virtualisation is not available. Distribution is under the Apache License 2.0, which constrains every dependency to a compatible licence. It also assumes that the operator controls and is trusted on the host, that the root encryption key is backed up, and that functions are written by people inside the organisation.

2.3 Use Case Diagram

Figure 2.2 shows the actors, the system boundary and the twenty-six use cases in five packages. Association lines are drawn beneath the ellipses, so a line that must cross a use case passes behind it. The primary actors are the **operator**, who holds the owner or admin role and manages clusters, nodes, accounts, credentials, ceilings, runtimes and updates; the **developer**, who creates namespaces and functions, deploys source and reads logs; the **caller**, any HTTP client; the **CI pipeline**, an automated client holding an API key; and the **scheduler**, which is the passage of time causing a due trigger to run. The **Docker Engine** is a secondary actor: it creates, starts and removes containers on the control plane's instruction.

2.4 Use Case Description

Every use case in Figure 2.2 is described below in the same form: what it is called, who takes part, whether it is primary to the system or secondary to another case, the flow of steps, the functional requirements it satisfies, and the smaller cases it draws on.

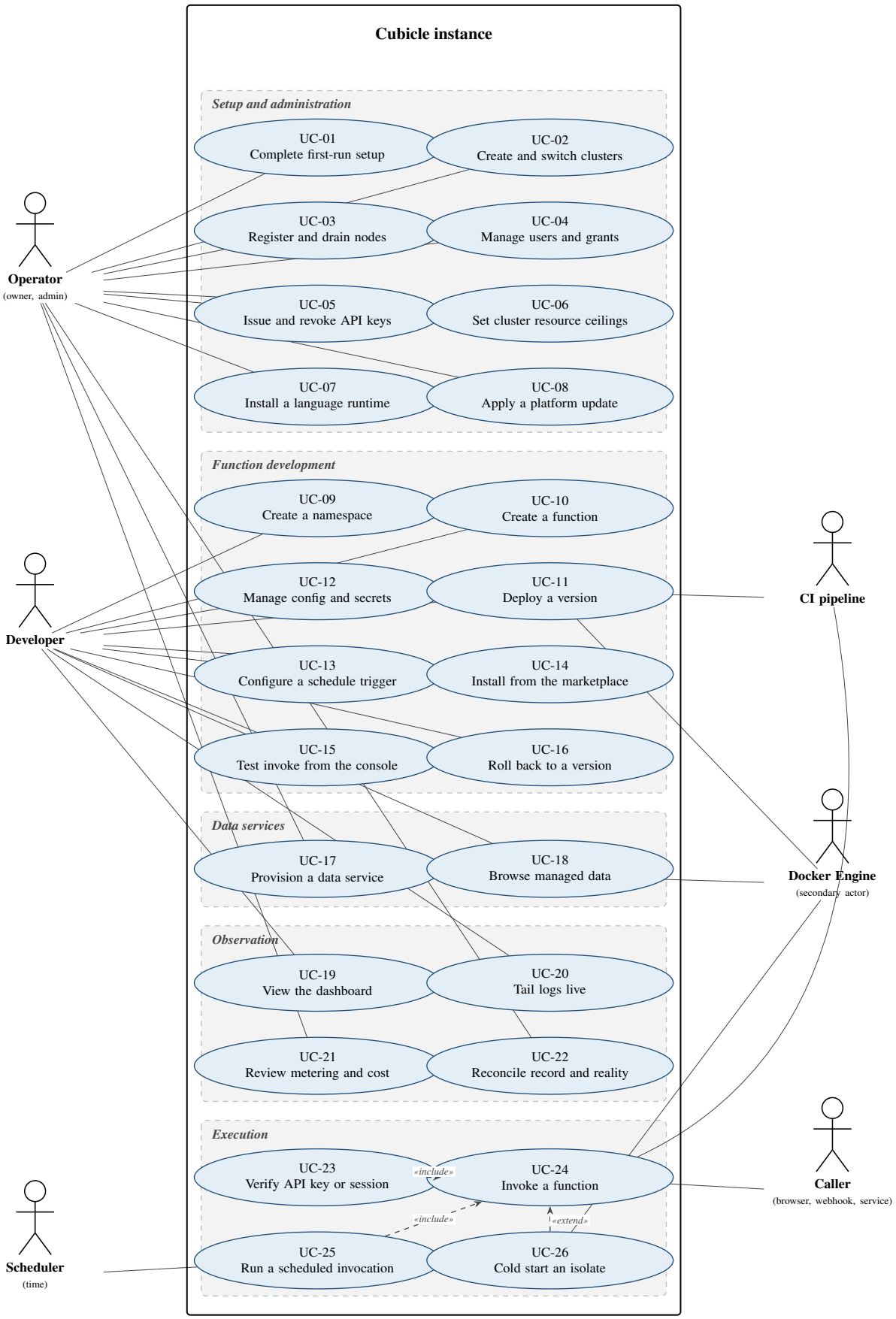


Figure 2.2: Use case diagram. Solid lines are associations between an actor and a use case; dashed arrows are the include and extend relationships. Verifying an API key is included by every authenticated invocation; a cold start extends an invocation only when the pool holds no warm container; a scheduled run reaches the same execution path as an HTTP call.

UC-01: Complete first-run setup

Use case:	Set up the instance
Actors:	Operator
Type:	Primary and essential
Description:	Open the instance → read the visible engines → give a name, an email, a password and a cluster → system hashes the password with argon2id → creates the owner, the first API key and the default cluster in one transaction → registers the local engine in the background.
Cross Ref.:	FR-01, FR-02
Use-Cases:	Register engine, Create cluster

UC-02: Create and switch clusters

Use case:	Manage clusters
Actors:	Operator
Type:	Primary and essential
Description:	Open Settings → create a cluster → system registers the local engine against it → the cluster appears in the switcher → selecting it scopes every later screen to that cluster.
Cross Ref.:	FR-03, FR-04
Use-Cases:	Register engine, Switch cluster

UC-03: Register and drain nodes

Use case:	Manage nodes
Actors:	Operator
Type:	Primary and essential
Description:	Add an engine by URL with client certificates → system reads its cores, memory and version → marks it schedulable → draining removes it from placement while its existing containers keep serving.
Cross Ref.:	FR-03
Use-Cases:	Read engine capacity, Place work

UC-04: Manage users and grants

Use case:	Manage accounts
Actors:	Operator
Type:	Primary and essential
Description:	Create an account → assign one of readonly, developer, admin or owner → grant the clusters it may address → a cluster with no grant answers exactly as one that does not exist.
Cross Ref.:	FR-05, FR-06
Use-Cases:	Assign role, Grant cluster

UC-05: Issue and revoke API keys

Use case:	Manage credentials
Actors:	Operator
Type:	Primary and essential
Description:	Create a key → optionally scope it to one cluster → system stores only a keyed hash and shows the token once → revoking it takes effect on the next request.
Cross Ref.:	FR-05, FR-24
Use-Cases:	Scope key, Revoke key

UC-06: Set cluster resource ceilings

Use case:	Set ceilings
Actors:	Operator (super admin)
Type:	Primary and essential
Description:	Open the ceilings tab → read the memory, processor and storage already committed → set the limits → save → every later allocation is checked against them at admission.
Cross Ref.:	FR-19
Use-Cases:	Read commitment, Refuse allocation

UC-07: Install a language runtime

Use case:	Install runtime
Actors:	Operator
Type:	Primary and secondary
Description:	Open the runtimes tab → choose a runtime → system builds its image on the node that needs it → the runtime becomes selectable when a function is created.
Cross Ref.:	FR-16
Use-Cases:	Build image, Rebuild image

UC-08: Apply a platform update

Use case:	Update the instance
Actors:	Operator
Type:	Primary and secondary
Description:	System reports that the upstream branch has moved → operator presses update → system pulls, rebuilds and restarts the control plane and console → reports the applied revision.
Cross Ref.:	FR-23
Use-Cases:	Check upstream, Restart services

UC-09: Create a namespace

Use case:	Create namespace
Actors:	Developer
Type:	Primary and essential
Description:	Open the playground → name the namespace → system checks the name is unused inside this cluster → the namespace becomes the first path segment of every function beneath it.
Cross Ref.:	FR-07
Use-Cases:	Check uniqueness

UC-10: Create a function

Use case:	Create function
Actors:	Developer
Type:	Primary and essential
Description:	Choose a namespace → name the function → pick a runtime and method → system creates it at the smallest resource configuration each control offers → scaffolds a handler.
Cross Ref.:	FR-07
Use-Cases:	Select runtime, Scaffold handler

UC-11: Deploy a version

Use case:	Deploy
Actors:	Developer, CI pipeline
Type:	Primary and essential
Description:	Save and deploy → validate the bundle → insert version $n{+}1$ as building → create a volume → copy the source in → install dependencies → mark ready and swap the current version → drain the old containers.
Cross Ref.:	FR-07, FR-08, FR-09
Use-Cases:	Validate bundle, Build version, Swap version

UC-12: Manage configuration and secrets

Use case:	Manage configuration
Actors:	Developer
Type:	Primary and essential
Description:	Open global environment → add a key and value → mark it secret if required → system encrypts it → the value is resolved at the next invocation with no redeploy.
Cross Ref.:	FR-13
Use-Cases:	Encrypt value, Resolve at invocation

UC-13: Configure a schedule trigger

Use case:	Schedule a function
Actors:	Developer
Type:	Primary and essential
Description:	Open the triggers tab of an independent function → give a cron expression and a timezone → system validates it and computes the next fire time → the scheduler claims and runs it when due.
Cross Ref.:	FR-18
Use-Cases:	Validate cron, Claim trigger

UC-14: Install from the marketplace

Use case:	Install a package
Actors:	Developer
Type:	Primary and secondary
Description:	Browse the registry → open a package → read its complete source → choose a namespace → system creates the function and leaves the declared configuration empty for the operator to fill.
Cross Ref.:	FR-17
Use-Cases:	Read registry, Create function

UC-15: Test invoke from the console

Use case:	Test a function
Actors:	Developer
Type:	Primary and essential
Description:	Open the test console → supply a body → system runs the function on the cluster → shows the status, the duration, the cold start flag and the handler output.
Cross Ref.:	FR-12
Use-Cases:	Invoke function, Read logs

UC-16: Roll back to a version

Use case:	Roll back
Actors:	Developer
Type:	Primary and essential
Description:	Open the versions list → choose an earlier version that built successfully → system points the current version at it → drains the containers of the version being replaced.
Cross Ref.:	FR-08
Use-Cases:	Swap version, Drain containers

UC-17: Provision a data service

Use case:	Provision database
Actors:	Operator
Type:	Primary and essential
Description:	Open data services → request PostgreSQL or Redis → system creates the container and volume, stores the password encrypted, and injects the connection into every function of the cluster.
Cross Ref.:	FR-15
Use-Cases:	Create container, Inject connection

UC-18: Browse managed data

Use case:	Browse data
Actors:	Developer
Type:	Primary and secondary
Description:	Open the database manager → choose a table → page, search and sort the rows → edit a row, inspect the structure, or run a statement in the SQL console.
Cross Ref.:	FR-15
Use-Cases:	Read table, Run query

UC-19: View the dashboard

Use case:	Read metrics
Actors:	Developer, Operator
Type:	Primary and essential
Description:	Open the overview → system aggregates the recorded invocations → displays the counters, the invocation histogram and the per function latency for the selected window.
Cross Ref.:	FR-20
Use-Cases:	Aggregate invocations

UC-20: Tail logs live

Use case:	Read logs
Actors:	Developer
Type:	Primary and essential
Description:	Open logs → system opens a server-sent event stream → handler output and control plane events arrive as they happen → a failed invocation carries the reason it failed.
Cross Ref.:	FR-21
Use-Cases:	Stream events, Report failure

UC-21: Review metering and cost

Use case:	Review metering
Actors:	Operator
Type:	Primary and secondary
Description:	Open cluster and metering → system totals the gigabyte seconds and egress from the invocation records → splits them by namespace → compares the total against public list prices.
Cross Ref.:	FR-20
Use-Cases:	Aggregate invocations, Charge back

UC-22: Reconcile record and reality

Use case:	Reconcile state
Actors:	Operator
Type:	Primary and secondary
Description:	Press scan → system enumerates the containers, volumes and networks the engine holds → compares them against the database → lists each difference with the repair it implies → applies the chosen repairs.
Cross Ref.:	FR-22
Use-Cases:	Enumerate engine state, Repair drift

UC-23: Verify an API key or session

Use case:	Authenticate
Actors:	System
Type:	Secondary and essential
Description:	Read the bearer token or session cookie → compare the keyed hash or look the session up in Redis → check the key's own cluster scope → check the account's grants → allow or refuse.
Cross Ref.:	FR-05, FR-06
Use-Cases:	Check scope, Check grant

UC-24: Invoke a function over HTTP

Use case:	Invoke a function
Actors:	Caller, CI pipeline
Type:	Primary and essential
Description:	Edge rewrites the URL → resolve cluster, namespace, function and current version → apply the admission guards → read configuration, secrets and session context → acquire a container → run the handler → apply context writes → release the container → record the invocation.
Cross Ref.:	FR-10, FR-11, FR-12
Use-Cases:	Verify credential, Cold start, Record invocation

UC-25: Run a scheduled invocation

Use case:	Run on schedule
Actors:	Scheduler (time)
Type:	Primary and essential
Description:	A trigger becomes due → scheduler claims it by updating the next fire time conditionally on the value it read → exactly one process wins → the function is invoked → the outcome is recorded on the trigger.
Cross Ref.:	FR-18
Use-Cases:	Claim trigger, Invoke a function

UC-26: Cold start an isolate

Use case:	Start a container
Actors:	System, Docker Engine
Type:	Secondary and essential
Description:	No idle container exists → create one from the runtime image with the version volume mounted read only and the limits applied → wait for the agent to report ready → hand it the request.
Cross Ref.:	FR-10
Use-Cases:	Create container, Check readiness

2.5 Activity Diagram

Three activities carry the behaviour that matters: turning source into a running version, deciding whether a request may run at all, and running it.

2.5.1 Deploying a version

Figure 2.3 traces a deploy. Two decisions determine the outcome. The first rejects a bundle that is not shaped like a function. The second decides whether the current version pointer moves. Everything before that move is preparation; the move itself is a single field update, which is what makes a deploy atomic from the caller's point of view and why a failed build cannot take a running service down.

2.5.2 Admission control and execution

Figure 2.4 lists every check a request passes before any resource is committed to it, in the order they are applied: the cheapest checks that can reject a request come first, and nothing that costs money happens until every guard has passed. Figure 2.5 continues from connector A. The three way decision at `pool.acquire` is the centre of the platform's resource behaviour. Reusing an idle container is the common case and costs an HTTP round trip; starting a new one is permitted only while the function is below its maximum and the cluster below its ceiling; at the ceiling the request queues rather than starting another container, which is what stops one busy function from consuming a whole node.

2.6 Sequence Diagram

Figure 2.7 shows first-run setup: the account, the API key and the cluster are created in one transaction, so an interrupted setup cannot leave an instance with a user and no cluster, and everything that can fail without invalidating it runs afterwards in the background.

Figure 2.8 is the interaction view of Figure 2.3, and shows two details the activity diagram cannot: the source is copied into a container that is already running, which makes it land inside the mounted volume and is why the same code path works against a remote engine; and the client is answered before the build finishes, so a slow dependency install never holds a connection open.

Figure 2.9 traces a cold invocation. The optional fragment holds the whole of the cold start cost, and a warm invocation is exactly this diagram with that fragment removed, which is the clearest statement of what the warm pool buys.

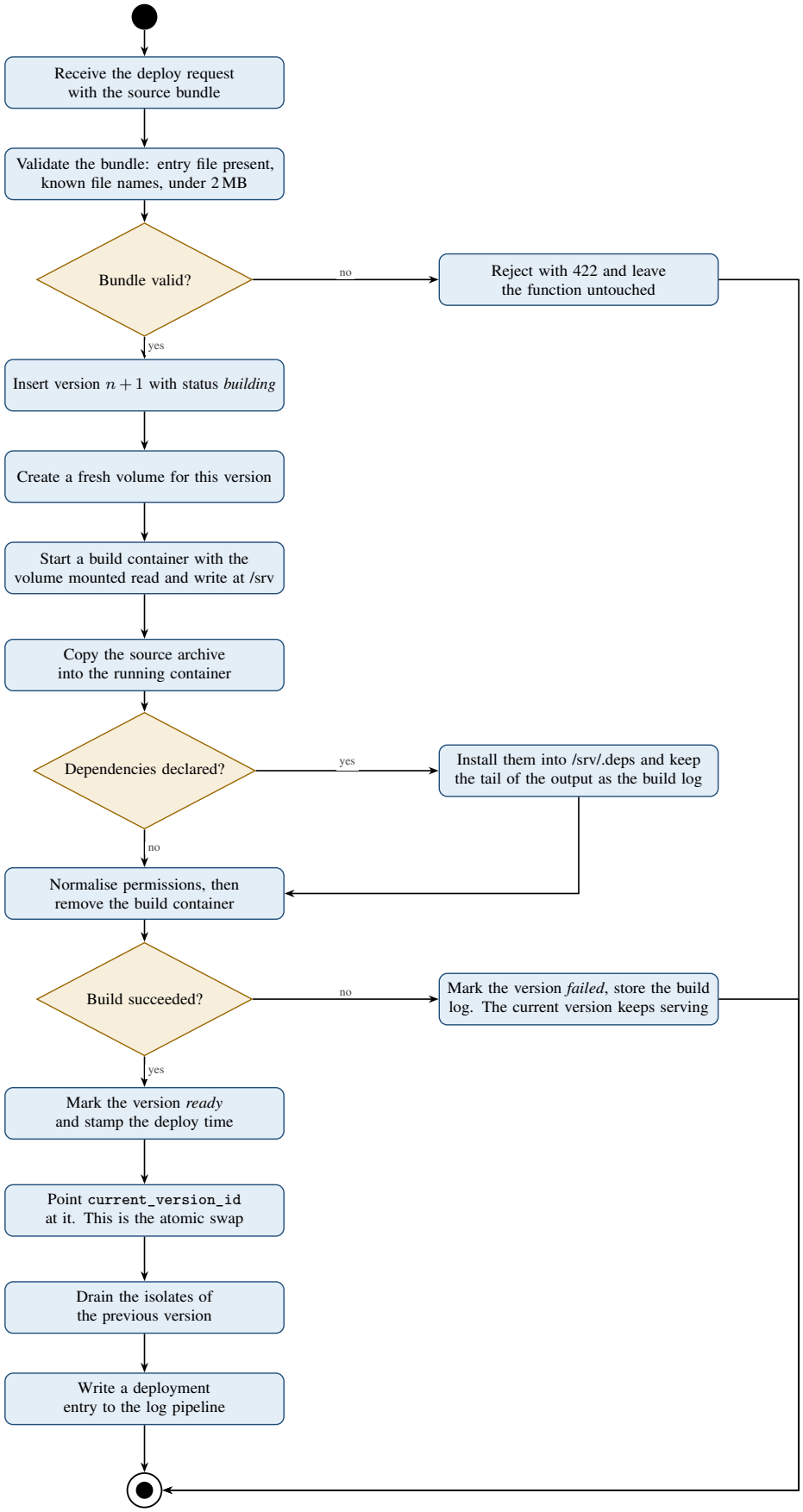


Figure 2.3: Activity diagram: deploying a function version. A failed build reaches the same end state as a successful one, but leaves the current version pointer untouched.

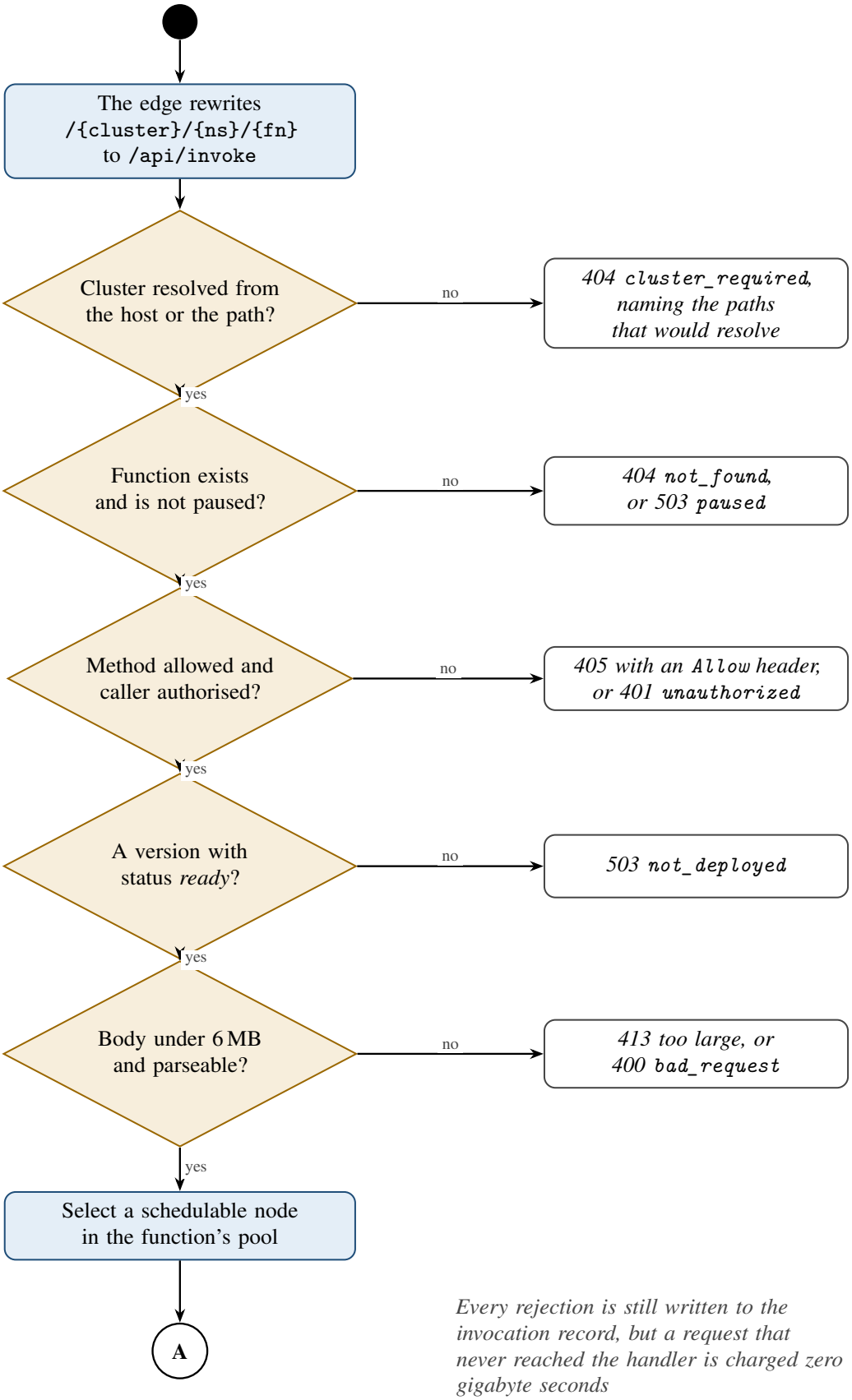


Figure 2.4: Activity diagram: admission control. Each guard has its own status code and message, so a caller is told which rule it broke. Control passes to Figure 2.5 at connector A.

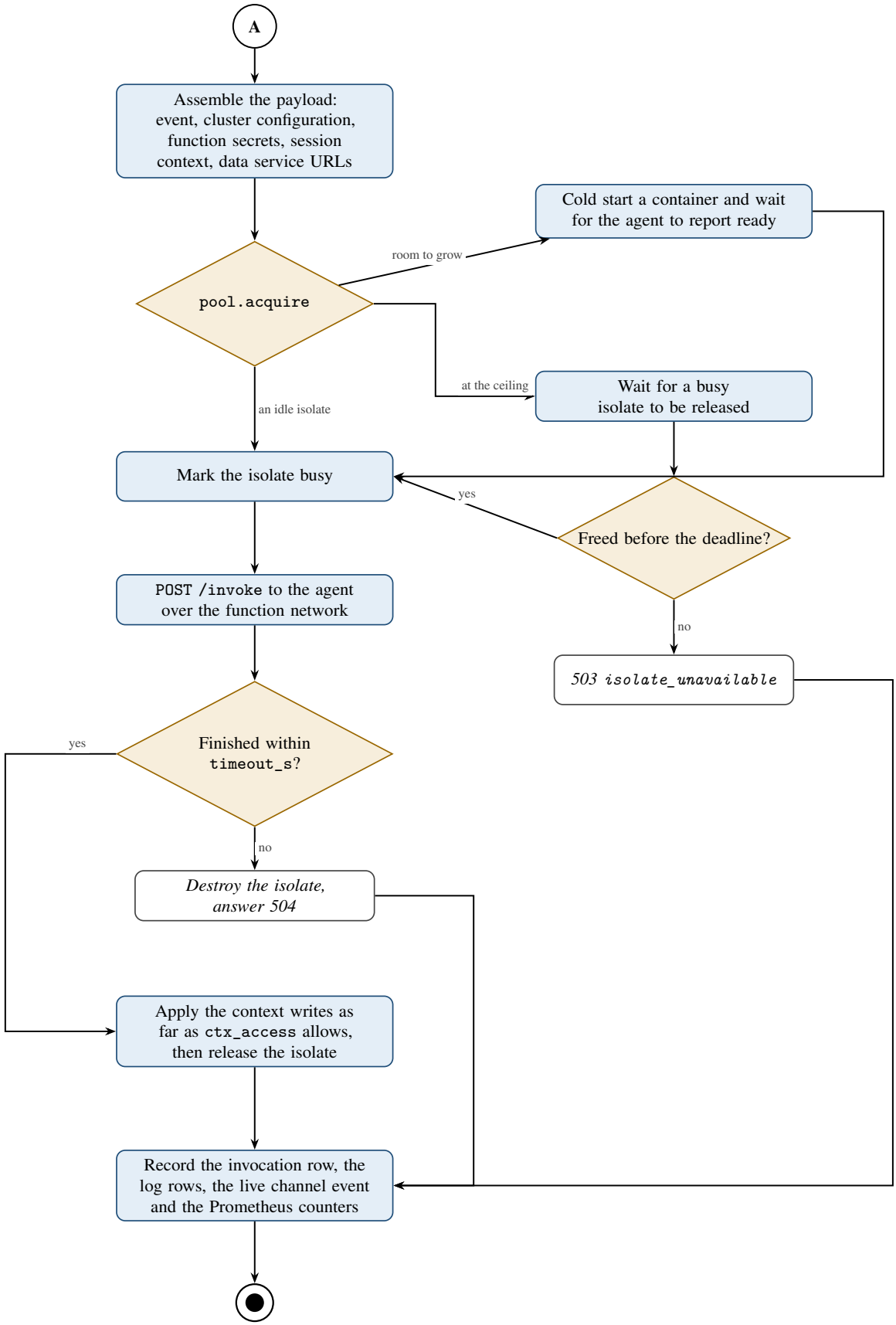


Figure 2.5: Activity diagram: scheduling and execution, continuing from connector A. A timeout destroys the container rather than returning it to the pool, because a handler thread that has not finished cannot be reclaimed safely.

2.7 Class Diagram

Figure 2.10 is the persistent model. Filled diamonds are compositions, implemented as cascading deletes: removing a cluster removes its namespaces, configuration, data services and its whole recorded history, which is what makes a cluster a real boundary rather than a label. Three constraints carry more weight than their size suggests. A namespace is unique per cluster rather than globally, without which the cluster boundary would be cosmetic. `Function.current_version_id` is what serves traffic, and building a new version does not touch it until the build succeeds. And `Trigger.next_run_at` is both the schedule and the lock: the scheduler claims a due trigger by updating that column conditionally on the value it just read, so two processes racing for the same trigger cannot both win.

Figure 2.6 is the behavioural view. `IsolatePool` is the only class holding mutable state about what is running, and every other component reaches containers through it; `FunctionSpec` is a value object carrying everything the pool needs, including the cluster's ceilings, so the pool needs no database access on the hot path.

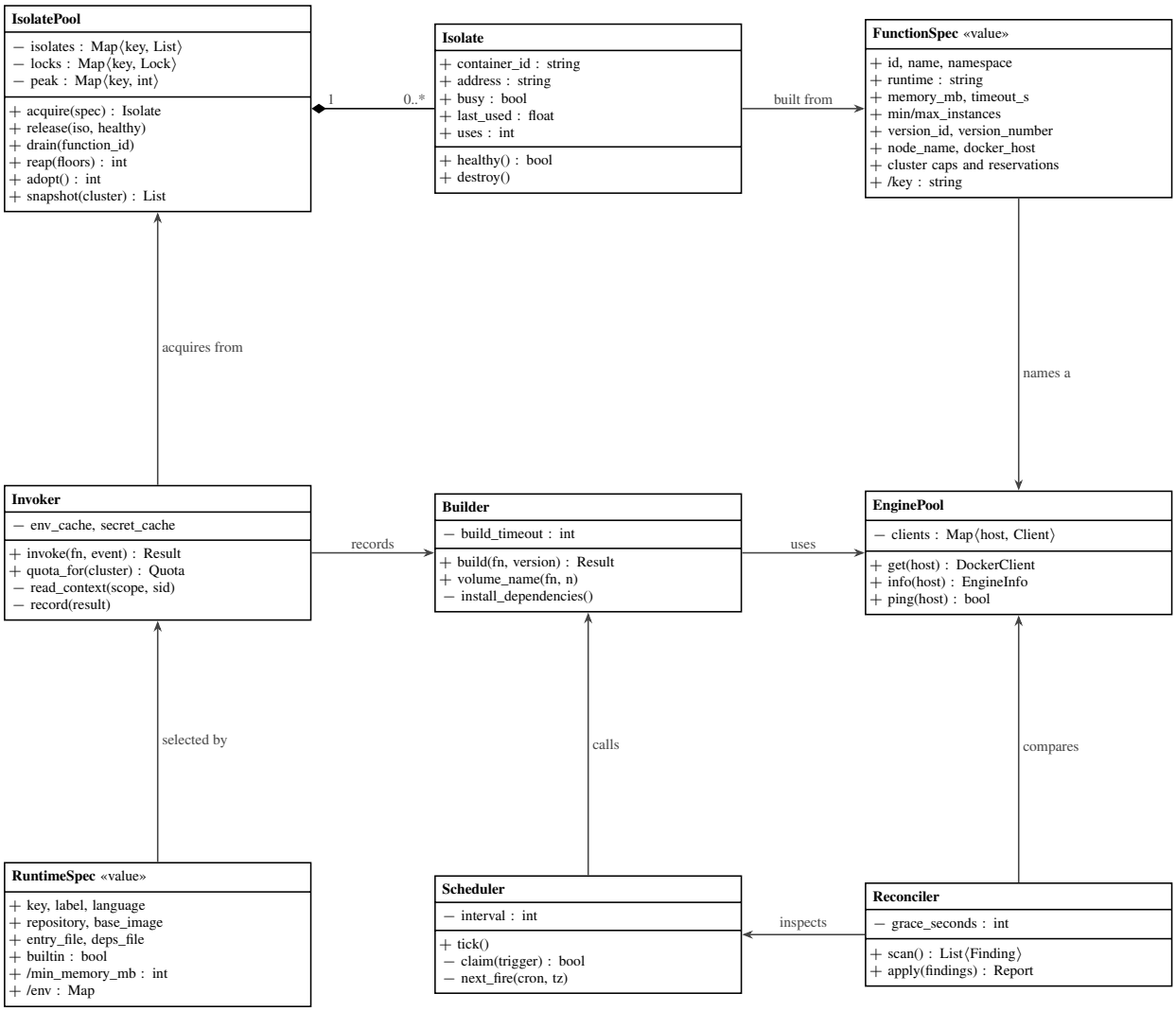


Figure 2.6: Class diagram: the control plane runtime. `RuntimeSpec` is plain data, which is what keeps the scheduler ignorant of any particular language: adding a runtime adds a row to a table, not a branch to the invocation path.

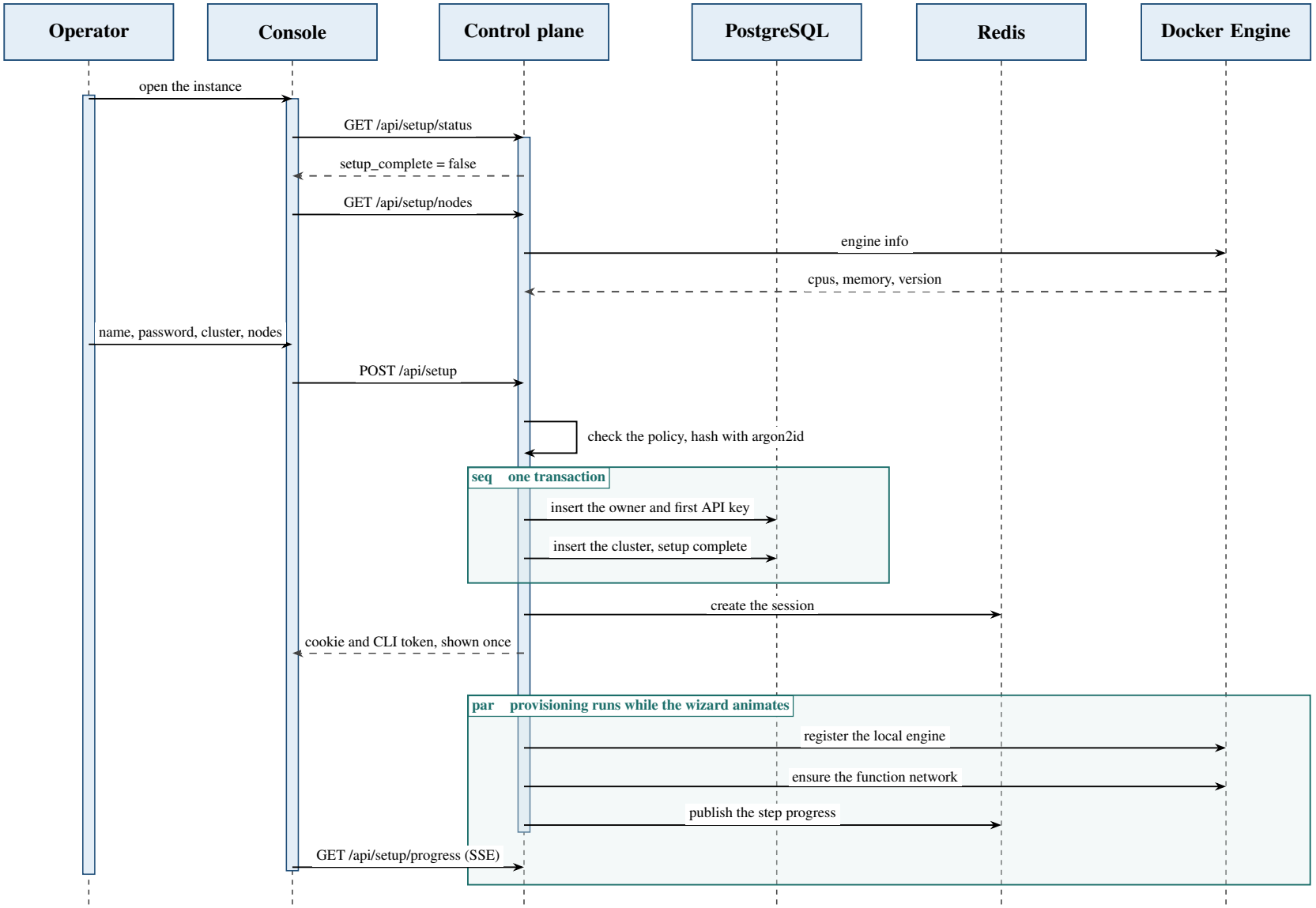


Figure 2.7: Sequence diagram: first-run setup. There is no registration anywhere in the product; the first person to open the console creates the only account the instance will have.

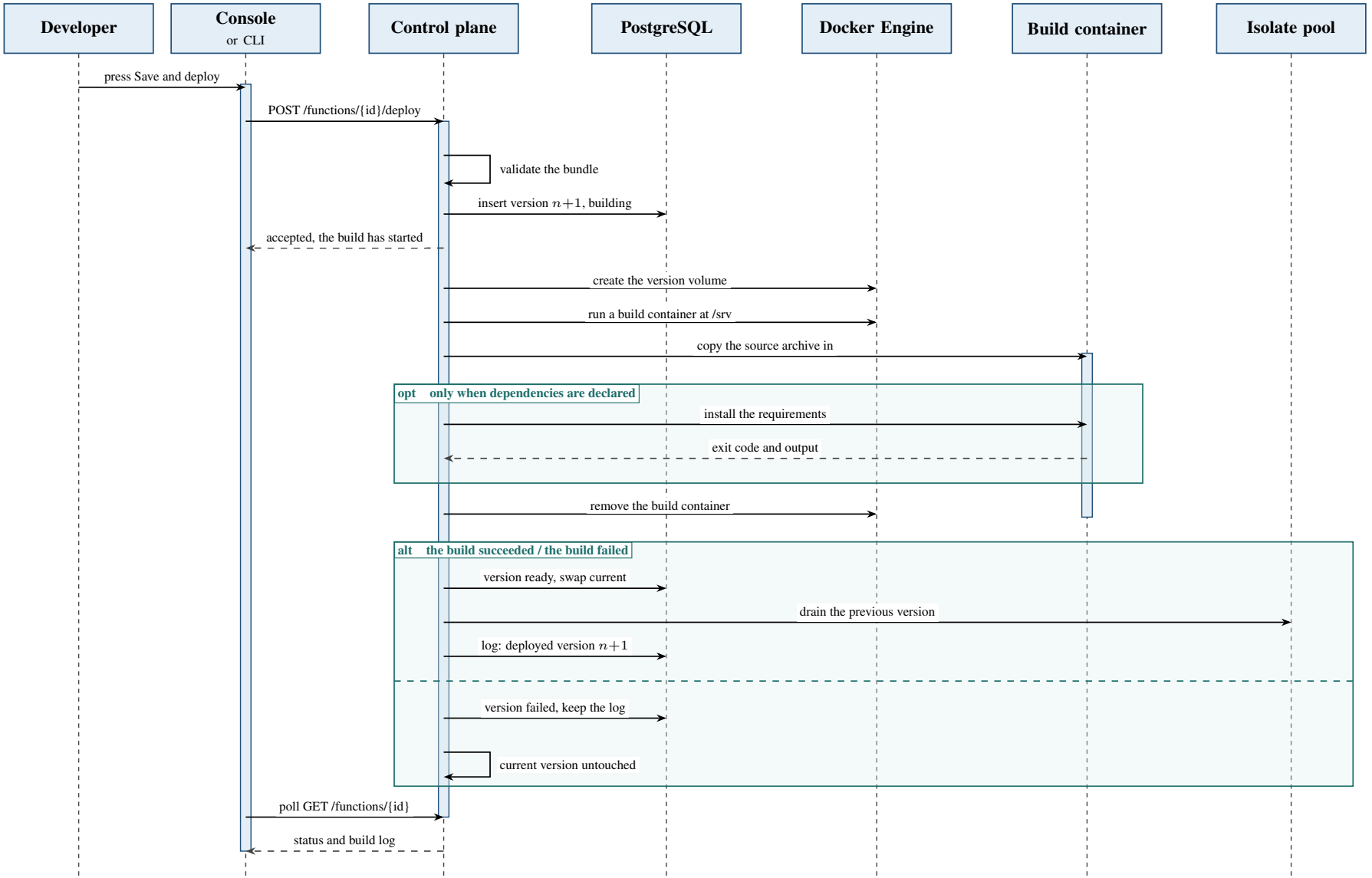


Figure 2.8: Sequence diagram: deploying a version. The alternative fragment holds the only two possible endings; in the failure branch the current version pointer is deliberately left alone.

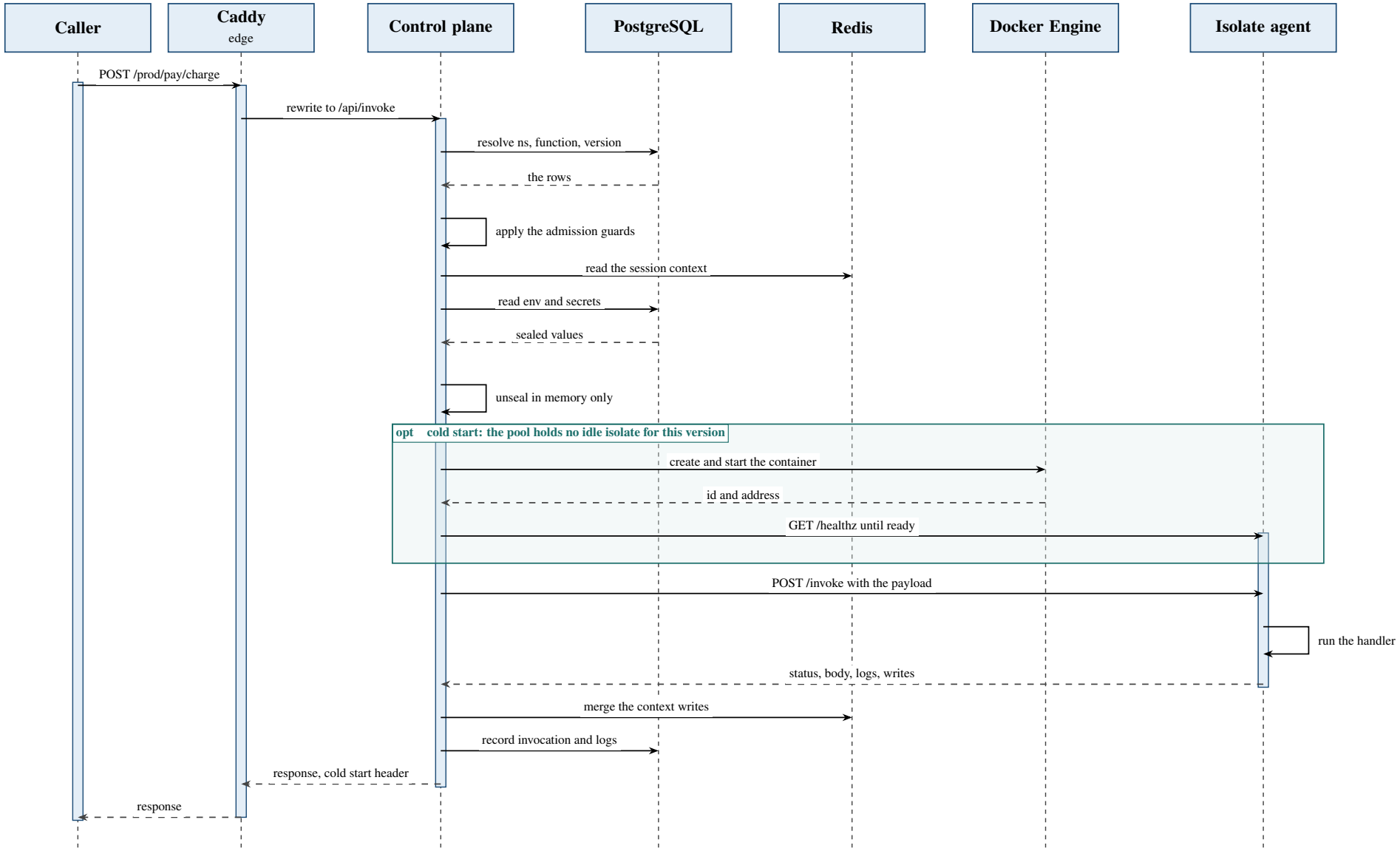


Figure 2.9: Sequence diagram: invoking a function on a cold start. Secrets are unsealed in memory and travel to the container in the invocation payload; they are never written to its filesystem or into the log pipeline.

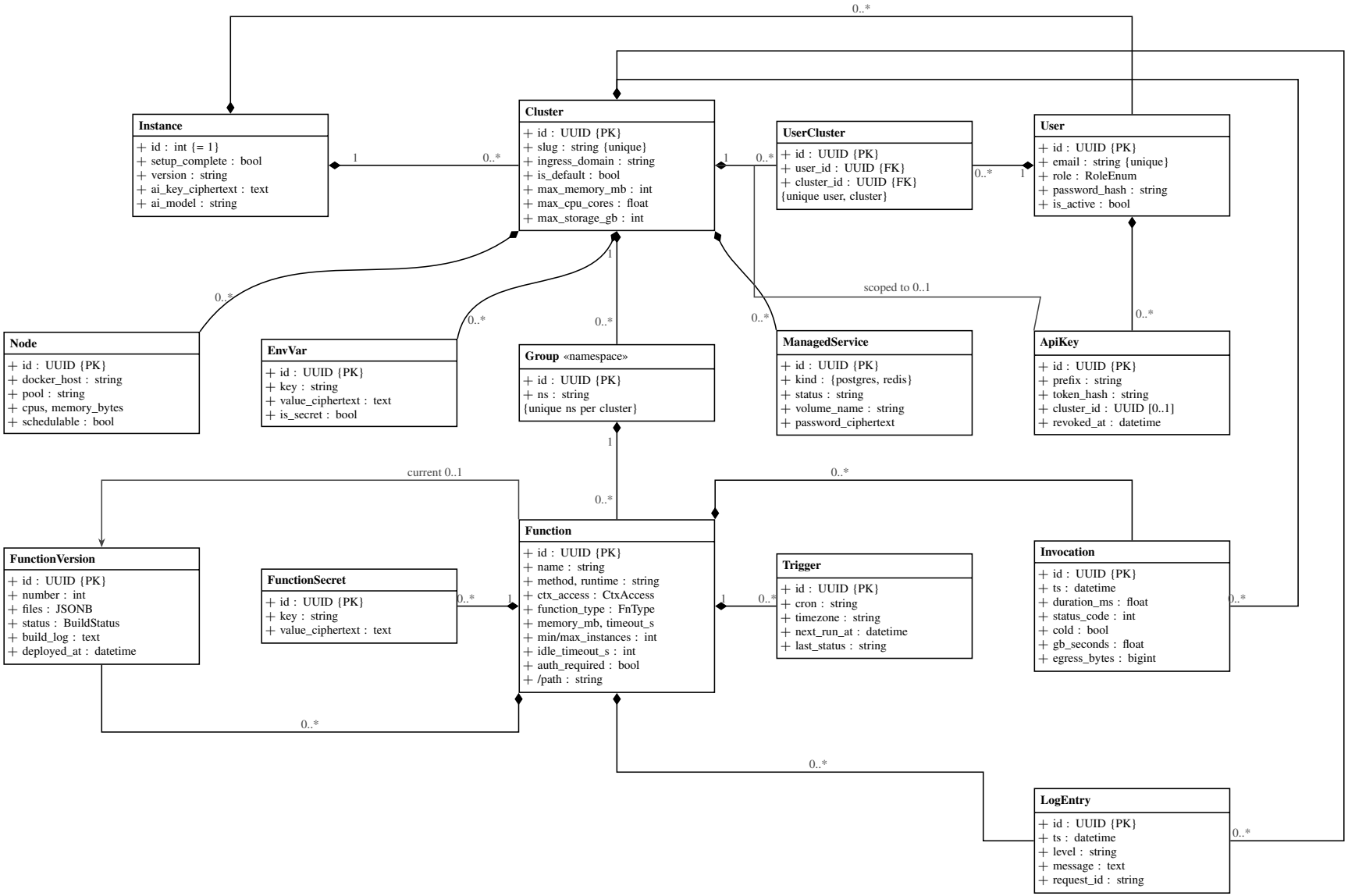


Figure 2.10: Class diagram: the persistent domain model. Only the defining attributes are shown.

CHAPTER 3

System Design and Implementation

3.1 System Architecture

3.1.1 Architectural style

Cubicle is a layered system with a clear separation between a stateless control plane and the stateful pieces it manages. Figure 3.1 shows the five layers. Three decisions shape everything above them.

First, **the deployable artefact is source code, not an image**. Platforms built on Kubernetes make the developer produce a container image, which needs a Dockerfile, a build pipeline and a registry. Here a deploy writes the source into a fresh volume and installs the declared dependencies into it, while the runtime image is shared and never rebuilt for an application change, so a deploy takes seconds, no registry exists in the system, and the developer never learns a container tool in order to ship a function.

Second, **the scheduler substrate is the Docker Engine API**. A node is any reachable container engine, local socket or remote, so there is no orchestrator to install and growing past one machine adds an engine rather than replacing the scheduler. Third, **the control plane keeps no durable state**: every fact lives in PostgreSQL or Redis, and what it holds in memory is a cache rebuilt at start-up by inspecting the engine and re-attaching to the containers it finds.

3.1.2 Deployment view

Figure 3.2 shows the containers on one host and the three networks that separate them. Only the edge container publishes a host port; isolates and managed data services are reachable from the control plane and from each other, and from nowhere else. The control plane mounts the container engine's socket, because creating containers is its job, and that mount is the trust boundary of the whole instance, as Section 5.1.3 says plainly.

3.1.3 Technology stack

The control plane is Python 3.12 with FastAPI, chosen for native asynchronous input and output, which suits a service whose work is almost entirely waiting on the engine and the database, and because the OpenAPI document is generated from the same annotations that validate requests, so the published contract cannot drift from the code. Persistence is PostgreSQL 16 through async SQLAlchemy 2.0 with Alembic, giving transactions across the whole of setup, JSONB for a version's file bundle and reversible migrations applied at start-up; Redis 7 holds revocable session tokens, the namespace context and the live log channel. The console is React 19 with TypeScript, Vite and Tailwind CSS, using CodeMirror 6 as the editor. Caddy 2 is the edge, because it issues and renews certificates with no configuration beyond a hostname. Execution is the Docker Engine API, identical for a local socket and a remote engine; scheduling uses croniter with named timezones; and the cryptography is argon2id, AES-256-GCM and HKDF.

3.1.4 Key mechanisms

Clusters. An instance holds one or more clusters, each owning its namespaces, configuration, data services, nodes and metrics. A namespace name is unique per cluster rather than globally, and every endpoint names its cluster either by a mapped hostname or by its slug in the path. There is deliberately no unqualified form: if a bare path were answered by the default cluster, promoting a different default would silently change what an existing URL addressed. A cluster an account has not been granted answers exactly as one that does not exist, because distinguishing the two would reveal which clusters it is kept out of.

Versioning. A deploy produces an immutable version whose files are stored as a JSONB document and whose built artefact is a Docker volume. The build is itself a container: the runtime image starts with the volume mounted for writing, the source is copied into the running container so it lands inside the volume, dependencies are installed into a private directory, and the container is removed. Nothing depends on a path

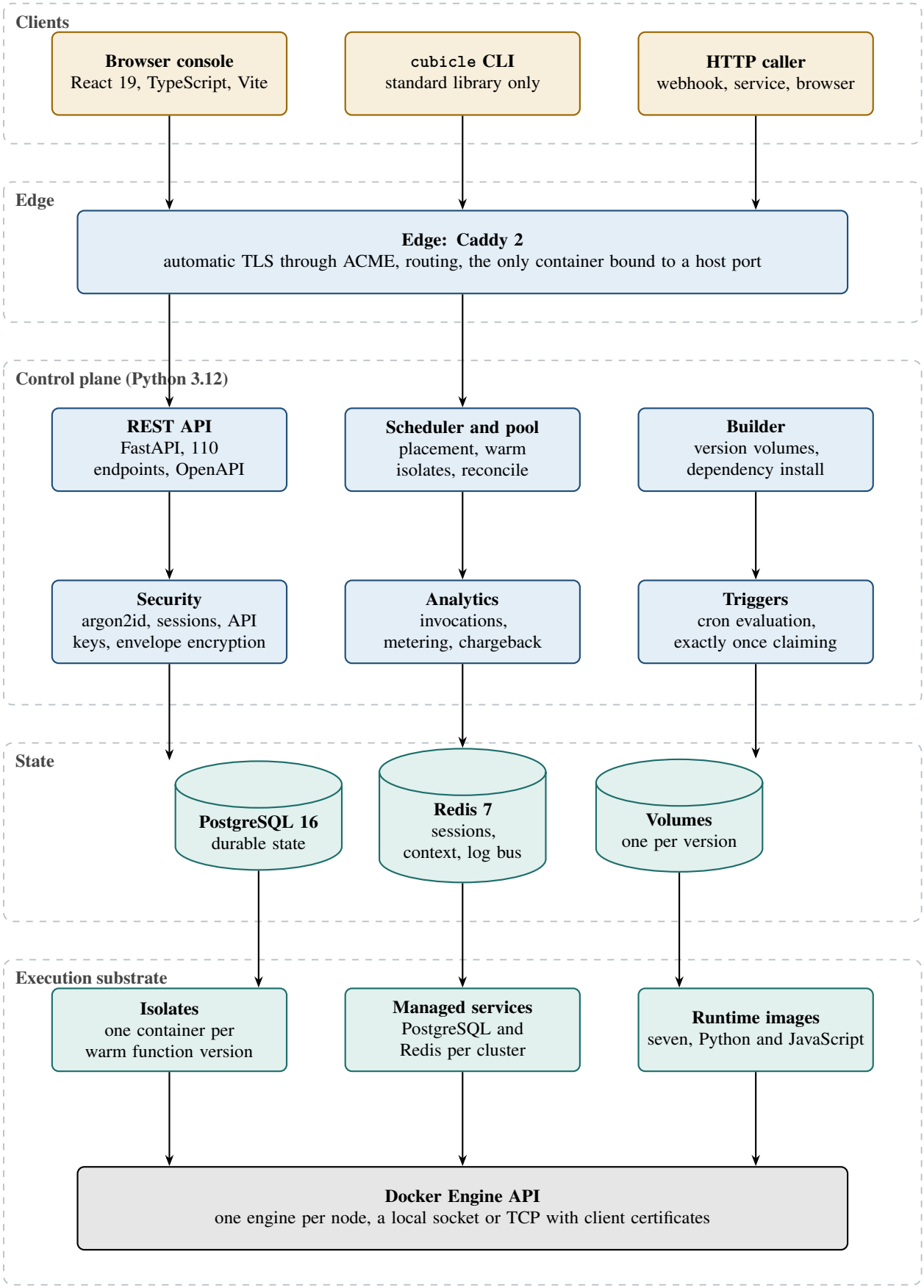


Figure 3.1: Layered system architecture. The control plane holds no durable state of its own, which is what allows it to be restarted at any moment without losing anything except the containers it had warm, and even those are re-attached rather than recreated.

shared with the host, which is why the identical code path builds on a remote engine. The atomic swap is a single field: until `current_version_id` moves, the previous version is what every request resolves to, and a failed build never moves it.

The isolate pool. Four properties were settled by measurement. Concurrency is isolates rather than threads: an isolate is marked busy and never given a second request, so with the per function cap set to two, twenty-four concurrent requests ran on exactly two isolates. Work is spread rather than stacked, because acquisition takes the least used idle isolate; sixty sequential requests across five isolates landed 21, 21, 22, 22 and 21. Reclamation happens in two stages, because spreading load keeps every timestamp fresh and an idle timeout alone would never shrink a pool a burst had grown; a twenty request burst against a cap of six went from six isolates to one in two and a half minutes. And restarts stay warm, because shutdown leaves isolates running and the pool re-attaches to those whose version is still current.

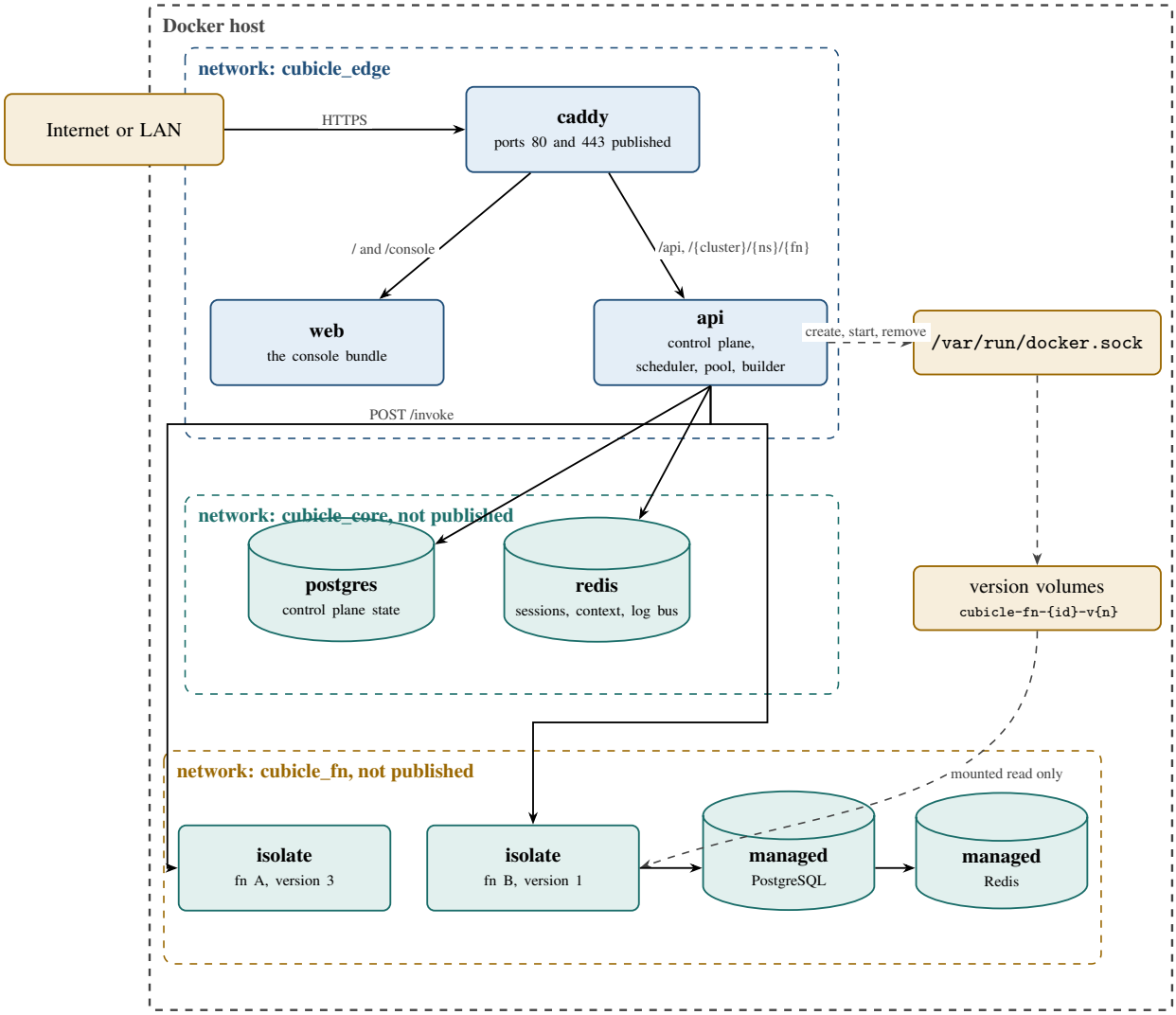
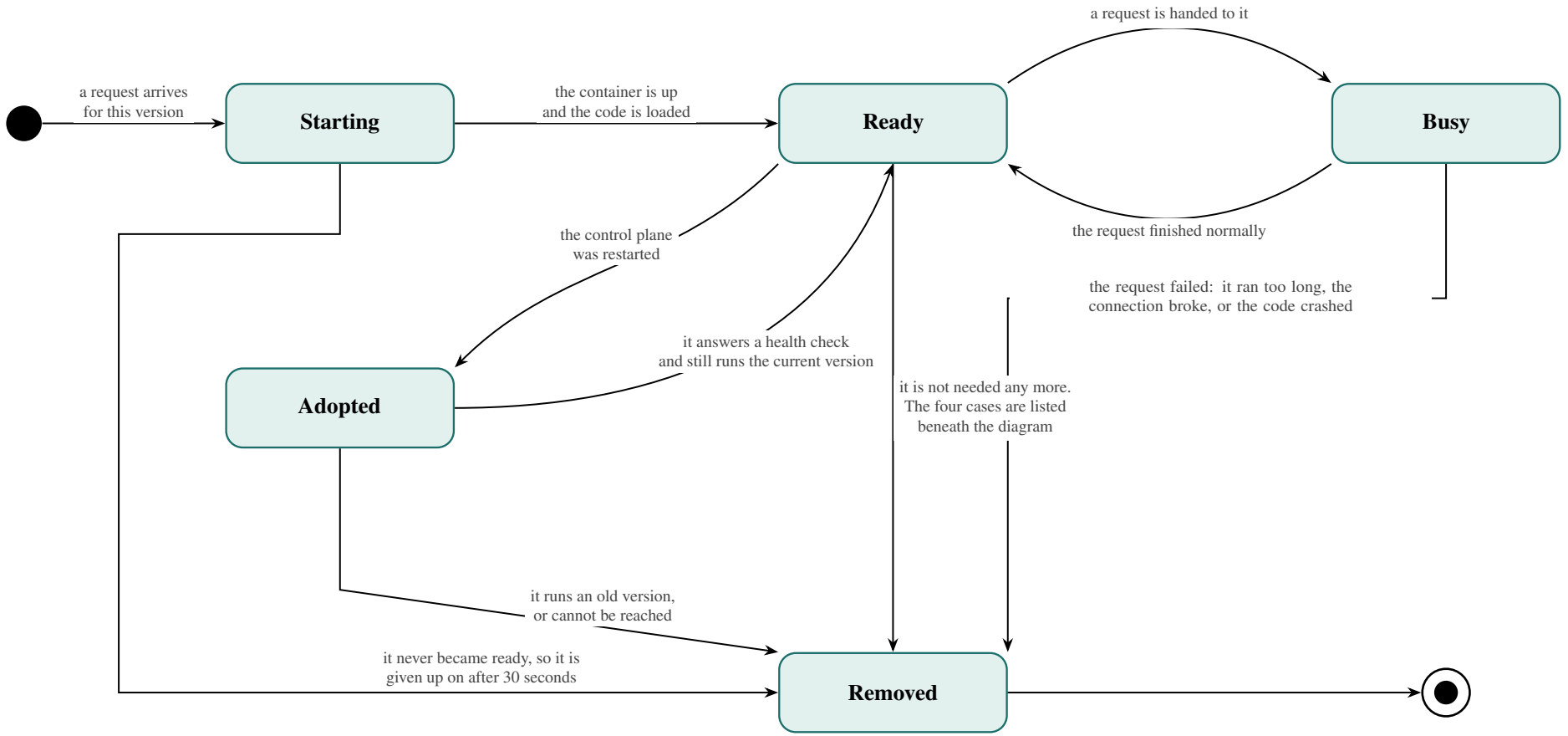


Figure 3.2: Deployment view: containers, the networks that join them, and what crosses each boundary. Function source never travels on a shared filesystem; each version gets its own volume, mounted read only.



When is a waiting container thrown away? A container sitting in *Ready* is removed in the four cases below. The first two are what *scale to zero* means here: a function nobody calls ends up costing nothing but a row in the database.

1. Unused for longer than the idle limit, and the function does not require one to stay alive.
2. More containers are running than recent traffic needs, so extras are given back one per pass.
3. A newer version has been deployed, so this container is now out of date.
4. The function was deleted, paused, or had its memory or timeout changed.

Figure 3.3: State machine: the life of an isolate. The transitions into *Removed* are together the definition of scale to zero. *Adopted* exists so that restarting the control plane does not cost every function a cold start.

The runtime registry. Nothing in the control plane knows one language from another: a runtime is described entirely by data, namely its image, base, entry file, dependency file and the smallest memory limit at which it serves. Seven are registered, three shipping with the instance and the rest installing on demand, and images are built rather than pulled because the agent has to be inside the image. The memory floors were measured: a Python isolate serves at 32 MB, starts but never answers at 24 MB, and does not start at 16 MB. One coupling caused a real defect: the agent is part of the runtime image, that image is tagged with the instance version, and the version never moved during development, so a change to the agent left an updated edge talking to a stale one. The repair was an explicit rebuild action in the console.

Resource governance. Each cluster carries a memory, processor and storage ceiling, where zero means none. They bound the per function settings rather than replacing them, so a function may ask for a gigabyte and still be refused because the cluster has none left. The accounting counts warm isolates, the reservations held by the cluster's own databases and the measured size of stored volumes, and enforcement happens at admission, before a container is created.

Triggers. A schedule is a cron expression and a timezone stored by name. The scheduler claims work by updating `next_run_at` conditionally on the value it read, so one row means this process owns the run and zero means another took it. That gives exactly once semantics without a distributed lock, a leader election or an external scheduler, verified with ten concurrent processes racing for one due trigger.

Observability. Every invocation writes a row recording duration, status, cold start flag, memory, gigabyte seconds, egress and node, and every displayed figure is an aggregate over those rows. Metering is honest about failure: a request that never reached the handler is charged zero, because billing an internal customer for a platform failure would make the metering page a lie. Logs go to PostgreSQL, retained for fourteen days, and onto a Redis channel the console consumes over server-sent events.

The handler contract. A function is one file, and both documented handler shapes run against the same runtime, so existing code does not have to be rewritten to move onto the platform.

```
from cubicle_context import Request, Context, env
from cubicle_db import postgres

def handler(req: Request, ctx: Context):
    body = req.json()
    api_base = env.get("PAYMENTS_API_BASE")     # resolved at invocation
    actor = ctx.get("actor") or {"tier": "anonymous"}
    with postgres.session() as db:
        db.execute("insert into charges (actor, amount) values (:a, :m)",
                   a=actor.get("id"), m=body.get("amount"))
    ctx.set("last_charge", {"amount": body.get("amount")})
    return {"ok": True, "actor": actor}
```

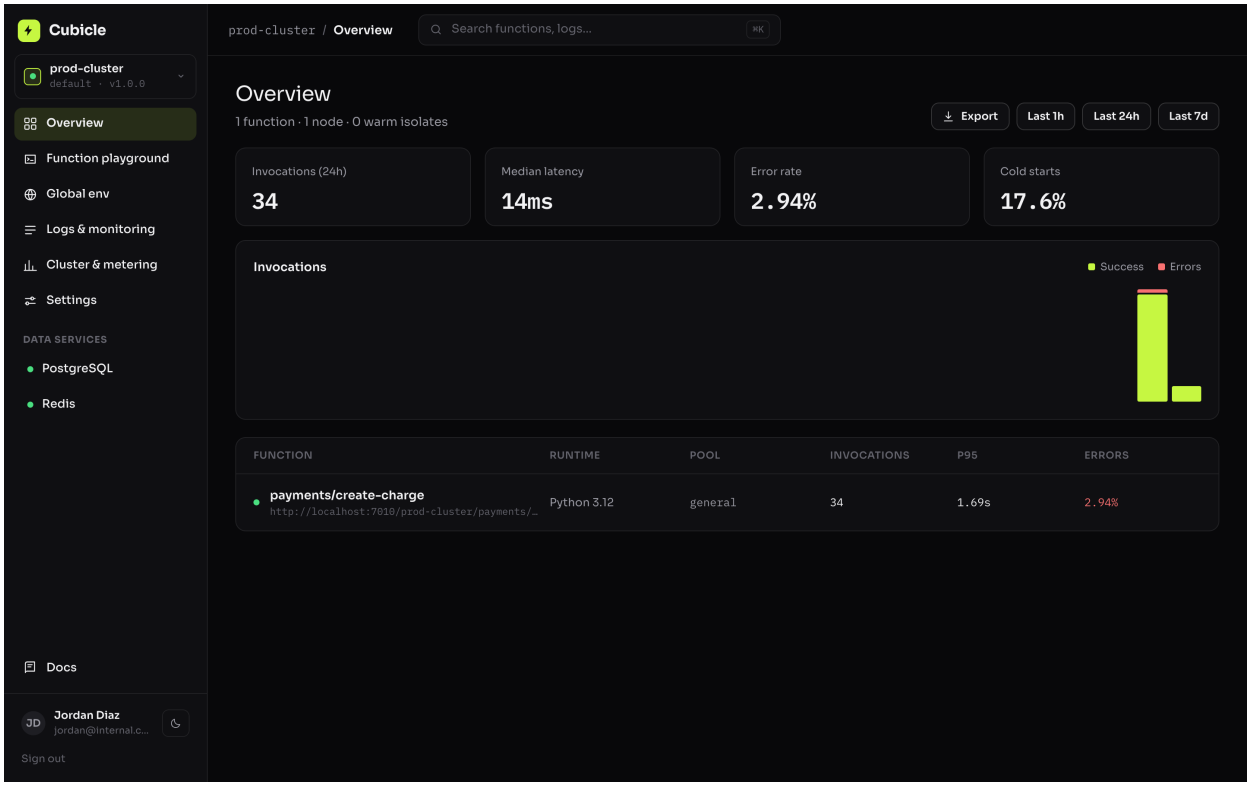
Listing 3.1: A handler using the bundled SDK. Cluster configuration is read at invocation, so changing it needs no redeploy; the session context is shared with the namespace.

3.2 Graphical User Interface

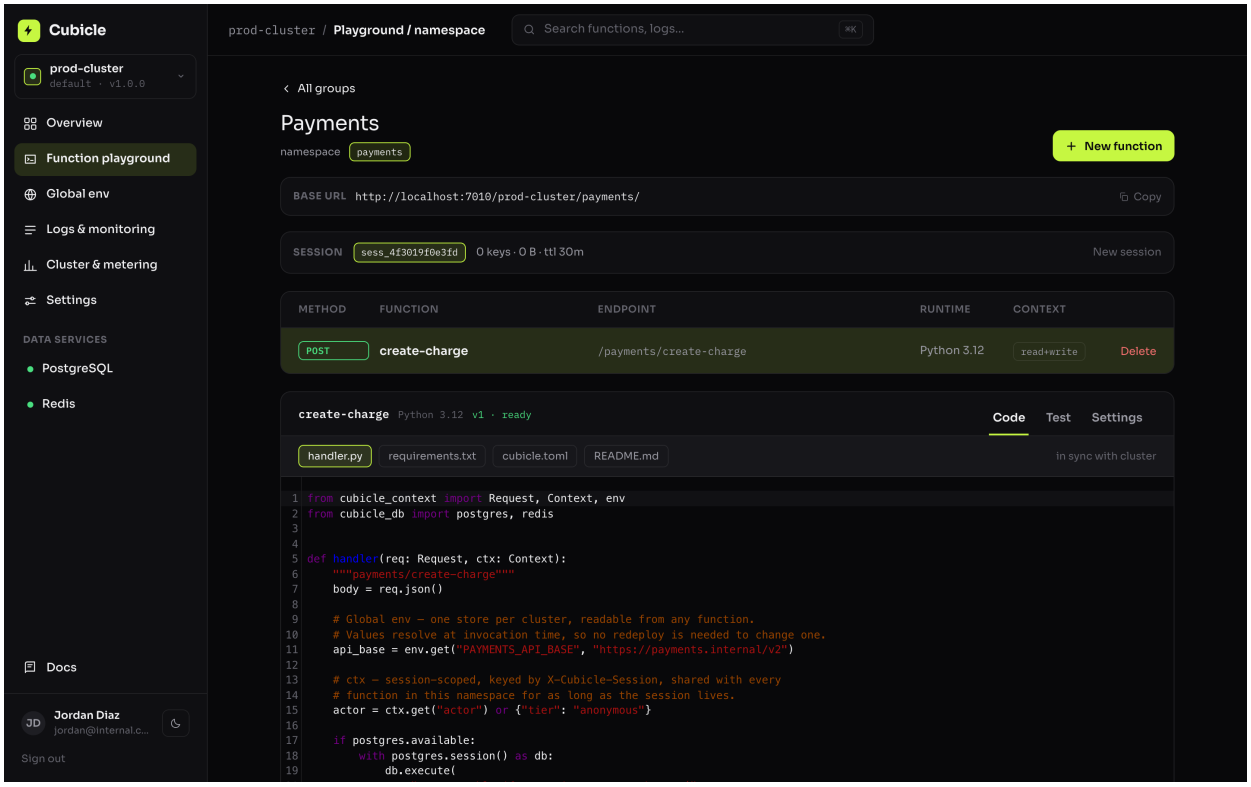
3.2.1 Structure and design decisions

The console is a single page application; Figure 3.7 maps its screens. Everything to the right of the console shell requires a session and an explicit grant on the selected cluster, and the cluster switcher is built from the clusters the account can reach, so a user is never shown the name of a cluster they cannot open.

Four decisions govern the interface. **Nothing applies on selection**: a settings form is a draft until saved, because an earlier version applied a change the moment a control was touched, so a mistake was live before it could be reconsidered. **One select component everywhere**, so a control behaves the same on every screen. **No third party request**: fonts are bundled, because a console that contacted an external host would contradict a product whose reason to exist is that data stays on the operator's hardware. **Live data is streamed or polled deliberately**: logs use server-sent events while the capacity meter polls, since a second stream would spend a scarce per origin connection on a number that changes slowly. Below the tablet breakpoint the sidebar becomes a drawer: a control may move, but none may become unreachable.

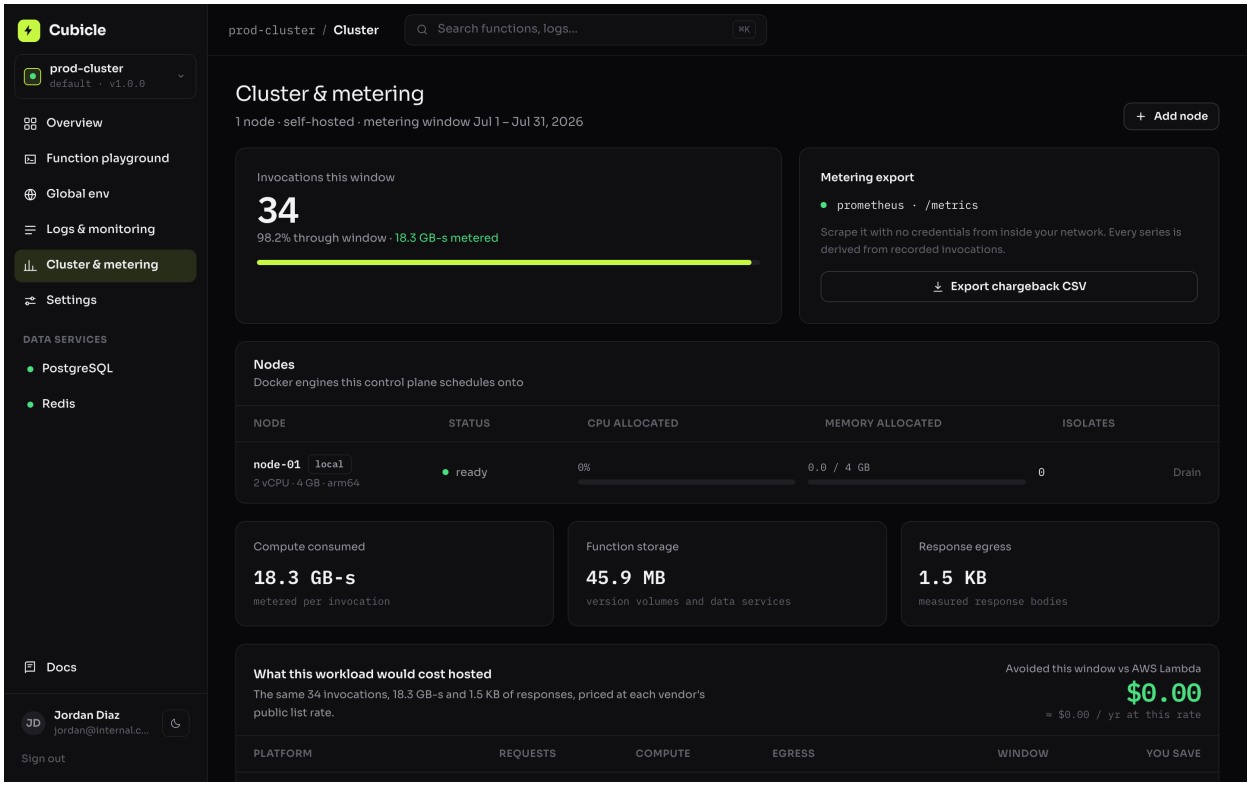


(a) Overview: counters, the invocation histogram and per function latency, every one aggregated from recorded invocations rather than sampled.

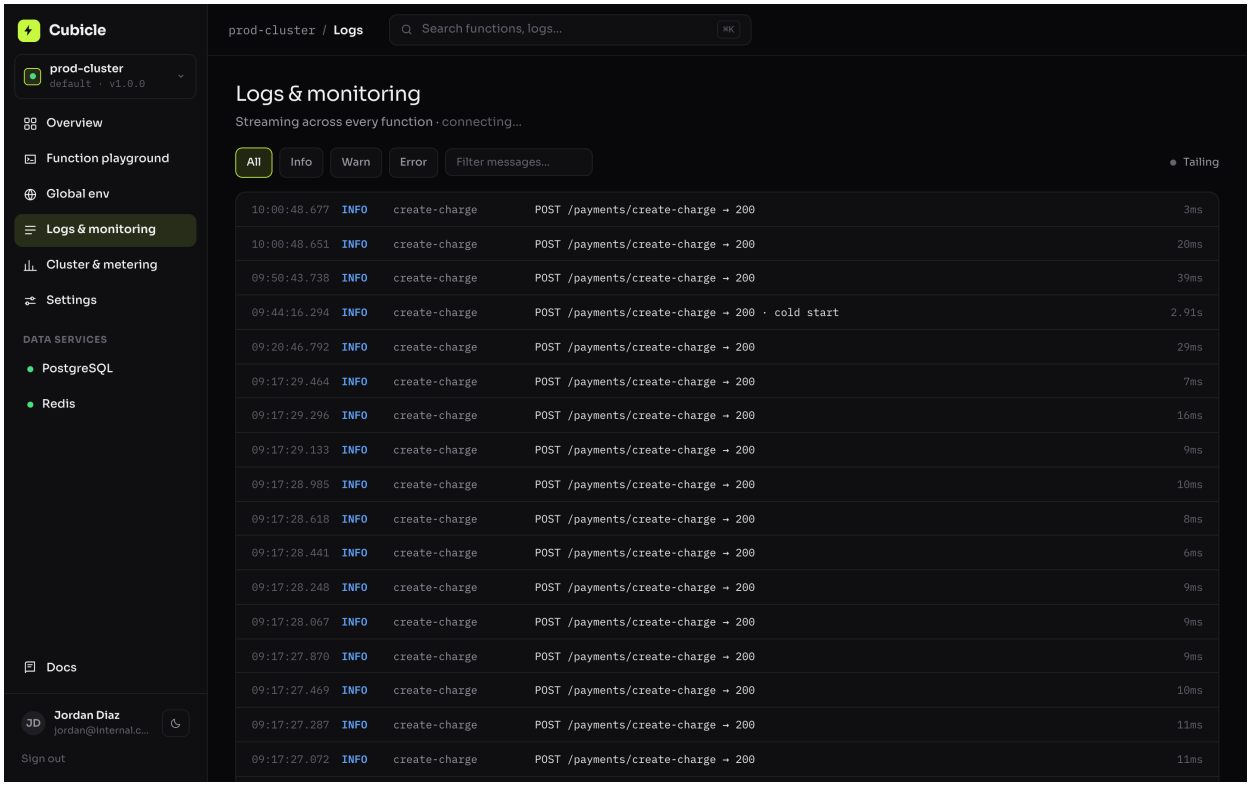


(b) Function playground: namespaces, a real editor, immutable versioned deploys, and a test console that runs the function on the cluster.

Figure 3.4: The console: the overview and the function playground.

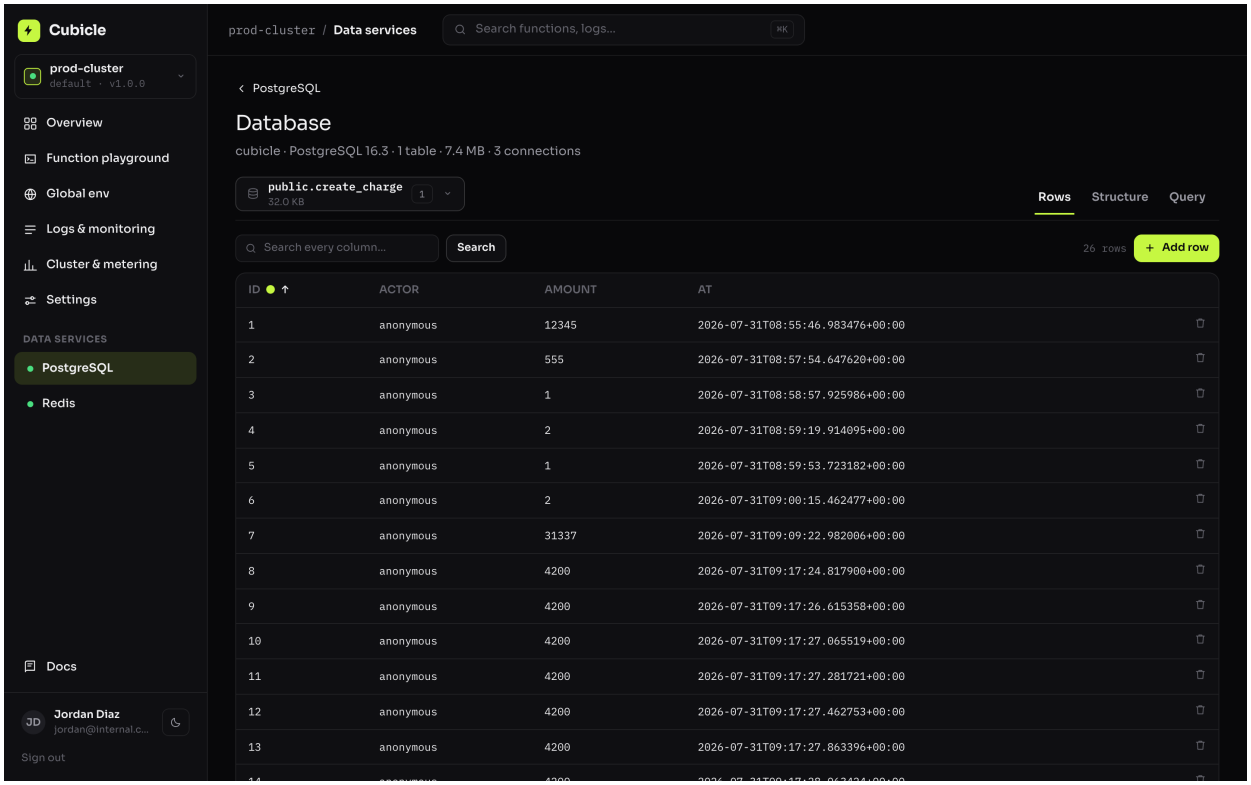


(a) Cluster and metering: node capacity and allocation, gigabyte seconds, measured egress and chargeback by namespace.

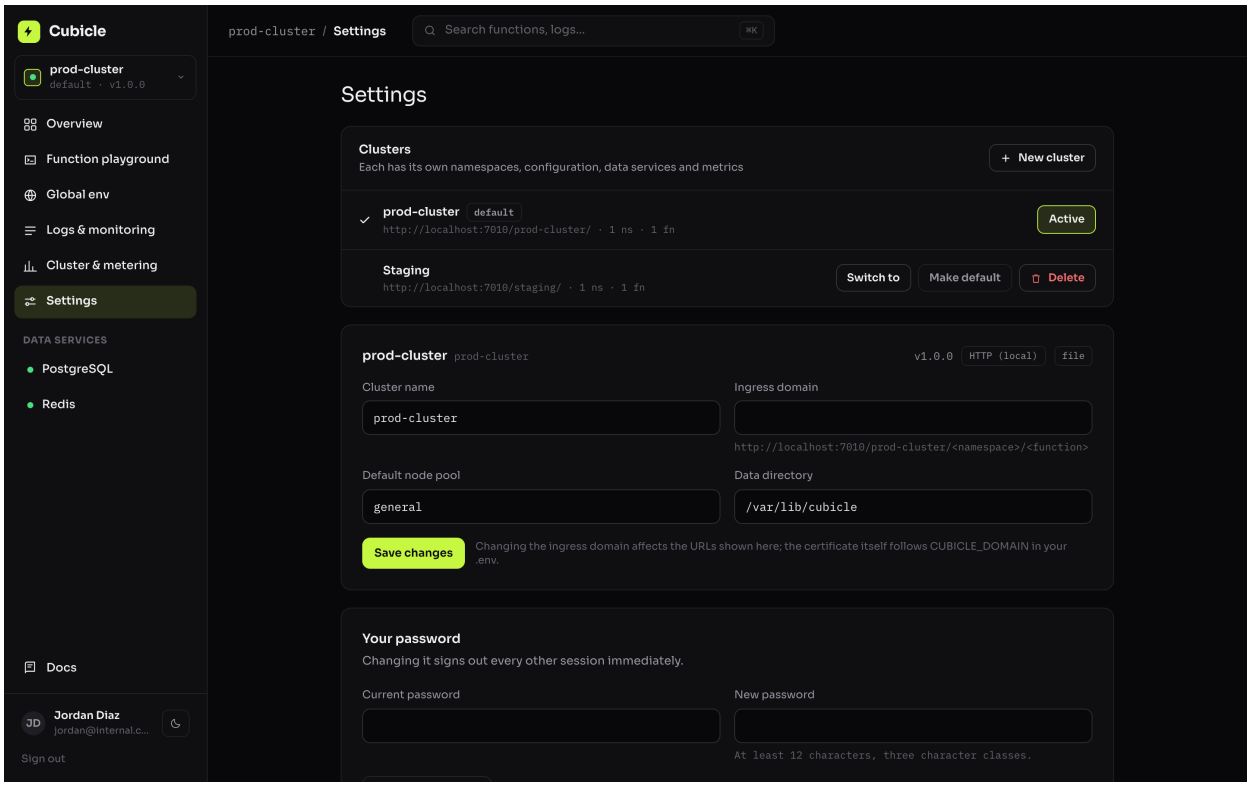


(b) Logs and monitoring: handler output and control plane events streamed live, each failure carrying the reason it failed.

Figure 3.5: The console: cluster metering and the live log view.

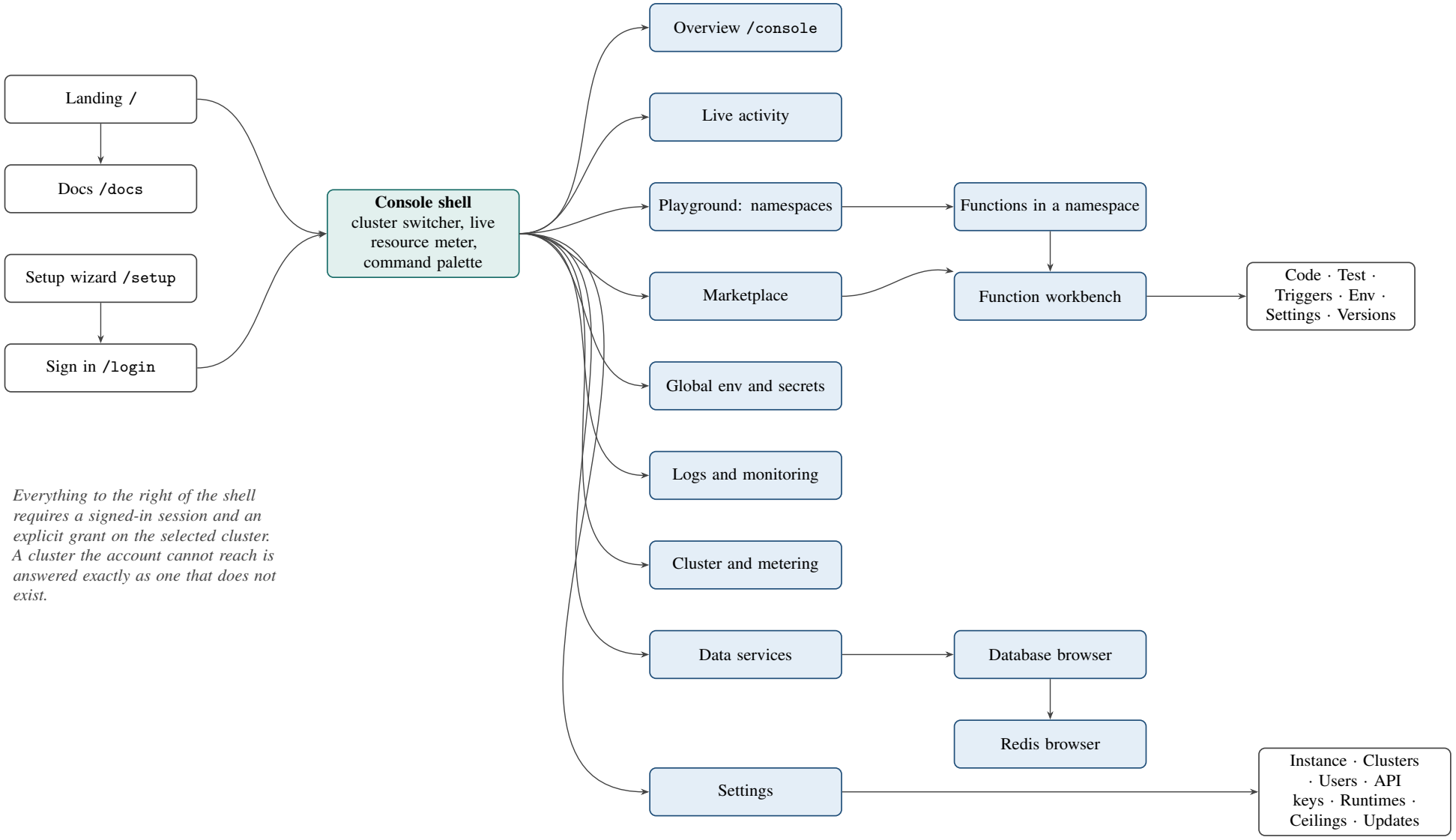


(a) Database manager for the cluster's provisioned PostgreSQL: paginated rows with search and sorting, table structure, and a SQL console.



(b) Settings, grouped into tabs: clusters, users, API keys, runtimes, resource ceilings and platform updates.

Figure 3.6: The console: the managed database browser and the settings tabs.



Everything to the right of the shell requires a signed-in session and an explicit grant on the selected cluster. A cluster the account cannot reach is answered exactly as one that does not exist.

Figure 3.7: Navigation map of the console. The shell carries the cluster switcher, the live remaining-capacity meter and the command palette, so those three are available from every screen.

CHAPTER 4

Project Planning and Requirements

4.1 Project Estimation and Planning

4.1.1 Size of the delivered system

Estimation has to start from a measurement of the product. Table 4.1 gives the size of each component, counted from the repository with generated files, dependency directories and build output excluded. Excluded from the count but part of the delivered work are the Docker Compose project, the runtime Dockerfiles, an installer of roughly 340 lines, the edge configuration and about 950 lines of documentation.

Table 4.1: Size of the delivered system

Component	Language	Lines	Contents
Console	TypeScript	19,045	17 pages, 15 shared components, the generated API client
Control plane	Python	13,366	19 routers, 15 runtime modules, domain model, cryptography, analytics
Automated tests	Python	1,561	127 test functions across 11 modules
Runtime images	Python, JS	1,073	The in-container agent and the bundled SDK for both languages
Migrations	Python	671	Alembic revisions applied at start-up under a lock
Command line client	Python	584	No dependency outside the standard library
Total		**36,300**	

4.1.2 Estimation by function points

Function point analysis sizes a system by what it does for its users rather than by how much code it took, which makes it the more honest of the two estimates here: the line count is a fact about the implementation, while the function point count is a fact about the specification. The components were counted from the API surface and the domain model, and complexity assigned using the standard weight bands.

Table 4.2: Unadjusted function point count

Component type	Low	Avg	High	Weights	Total
External inputs	12	26	9	3 / 4 / 6	194
External outputs	4	9	5	4 / 5 / 7	96
External inquiries	18	21	6	3 / 4 / 6	174
Internal logical files	4	8	3	7 / 10 / 15	153
External interface files	2	2	0	5 / 7 / 10	24
				Unadjusted function points	**641**

The two external interface files are the container engine's API and the marketplace registry index, both data the system reads but does not own. The internal logical files are the fifteen tables, grouped so that a table and its dependent detail count once. The fourteen general system characteristics were each rated from 0 to 5, giving a total degree of influence of 58; the high ratings are data communications, online data entry, online update, installation ease and operational ease, and the low ones are heavily used configuration and transaction rate, since the target is a single modest host.

$$\mathrm{VAF} = 0.65 + 0.01 \times 58 = 1.23, \qquad \mathrm{FP} = 641 \times 1.23 \approx 788$$

At the commonly cited band of 10 to 20 function points per person month, 788 function points implies roughly 39 to 79 person months.

4.1.3 Estimation by COCOMO

The basic COCOMO model [10] estimates effort and schedule from size alone. The project fits the organic mode: a small team, a familiar domain and few externally imposed interface constraints. With $\mathrm{KLOC} = 36.3$ the model predicts:

$$E = 2.4 \times 36.3^{\,1.05} \approx 104 \text{ person months}, \qquad D = 2.5 \times E^{\,0.38} \approx 14.6 \text{ months}, \qquad P = E/D \approx 7 \text{ people}.$$

The observed outcome was 1.1 person months, eight working days and a team of five; COCOMO is therefore out by roughly 95:1 on effort and 55:1 on schedule.

Presented without context, “104 person months” next to “eight days” would be a calculation rather than an estimate. The gap is explained by four factors:

1. **Calibration and tooling.** The model predates modern tooling and does not account for a team using a large language model as a pair, so it can be informative for relative comparisons but not for absolute figures.
2. **KLOC as a size proxy.** The line count is dominated by declarative code; roughly 19,000 lines are typed component markup.
3. **Coordination overhead.** Five people in one room have ten communication paths, whereas COCOMO’s nominal seven-person team implies twenty-one.
4. **Requirements effort.** The domain was understood before construction, so requirements elicitation was largely replaced by the survey in Section 1.4.

The value in running the models was the decomposition: the function point count makes the specification’s breadth visible, and it is the breadth rather than the line count on which this project should be judged.

4.1.4 Schedule and risk

Figure 4.1 is the actual schedule, taken from the version control history rather than drawn in advance. The unit is a day because that is the resolution at which work happened, and the three days carrying no commit are shown as idle rather than filled in. Two features of the plan are visible. Testing runs across the whole span rather than sitting as a phase at the end, which is what an incremental model requires, since an increment that has not been verified is not deliverable. And tasks overlap, because work that became blocked was set down and picked up later rather than held onto.

Task	Duration	Day 1 Fri	Day 2 Sat	Day 3 Sun	Day 4 Mon	Day 5 Tue	Day 6 Wed	Day 7 Thu	Day 8 Fri
Increment 1: core platform	1 d								
Requirements analysis and architecture	0.4 d								
Control plane, data model, migrations	1 d								
Isolate pool, builder, invocation path	0.9 d								
Console shell, playground and editor	0.8 d								
Increment 2: multi-tenancy	1.6 d								
Clusters, access control and ceilings	1.4 d								
Managed data services and browsers	1.3 d								
Increment 3: operability	1.3 d								
Observability, metering, reconciliation	1.3 d								
Increment 4: language reach	1 d								
Runtime registry, JavaScript, marketplace	1 d								
Increment 5: automation and repair	2 d								
Triggers, settings rework, traffic report	1 d								
Defect repair, documentation, release	1 d								
Verification and testing	8 d								

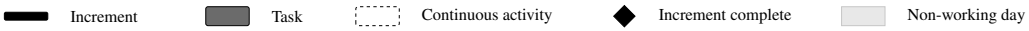


Figure 4.1: Project schedule, reconstructed from the version control history. The five increments appear as clusters of related bars.

Table 4.3: Risk register, with the mitigation actually applied

ID	Risk	Prob	Impact	Mitigation and outcome
R1	Container start-up too slow for a request path	Med	High	A warm pool with reuse between requests. Cold start measured at a few hundred milliseconds and reported to the caller. Retired in increment 1.
R2	Containers are the wrong boundary for the security claim	High	High	The claim was narrowed to match the mechanism rather than the mechanism overstated. Accepted and documented in Section 5.1.3.
R3	Recorded state drifts from what the engine runs	Med	Med	A reconciler that compares the two and repairs on request, verified by manufacturing real drift rather than by mocking it. Retired in increment 3.
R4	A schedule fires twice, or not at all	Med	High	Claim by conditional update, verified with ten concurrent processes racing for one trigger. Retired in increment 5.
R5	A change inside the runtime image ships only halfway	Med	High	Occurred. Repaired by adding an explicit rebuild action; the durable fix is listed in Section 5.2.
R6	Loss of the root key makes every secret unrecoverable	Low	High	Stated as intended behaviour rather than treated as a defect, with backup guidance in the operating documentation.
R7	Scope grows faster than the schedule	High	Med	The incremental model itself: each increment had to leave a deployable system, so growth was absorbed by adding an increment rather than extending one.

4.2 Hardware and Software Requirements

4.2.1 Development environment

Development used ARM64 workstations with 16 GB of memory running Docker Desktop, with Python 3.12 and Node.js 22 toolchains, Visual Studio Code, Git with the repository published to GitHub, ruff for Python linting, TypeScript in strict mode, and pytest with pytest-asyncio for the test suite. The API was explored through the generated OpenAPI document and `curl`. This report was written in LaTeX with TikZ.

4.2.2 Server and client requirements

The figures below are what the platform needs before any function is deployed; function memory is additional and bounded by the cluster ceiling. Two arithmetic points follow from the measured floors: a machine with 2 GB can hold on the order of thirty concurrent warm Python isolates after the platform's own footprint, and because idle functions are reclaimed to zero, the number of *deployed* functions is bounded by storage rather than by memory.

Table 4.4: Server requirements

Requirement	Minimum, recommended	Note
Processor	2 cores, 4 cores; x86-64 or ARM64	Cold start time is dominated by the processor
Memory	2 GB, 8 GB	The five platform containers hold roughly 900 MB between them
Storage	20 GB, 60 GB solid state	Control plane database, one volume per function version, any provisioned data service
Network	One reachable address; a domain name	Ports 80 and 443 must be reachable for the certificate challenge
Linux kernel	5.10 or later	Namespaces and control groups v2, on which the containment settings depend
Docker Engine	24 or later, with Compose v2	The scheduling substrate
Git	Any current version	The installer clones the repository; the self update reads the upstream branch
Supplied as containers	PostgreSQL 16, Redis 7, Caddy 2, Python 3.12, Node.js 22	Not installed separately; they come with the Compose project

The console requires only a current browser with JavaScript enabled and support for server-sent events. The command line client requires Python 3.9 or later and has no dependency outside the standard library, so it installs on an air-gapped jump host. Invoking a function requires nothing but an HTTP client.

CHAPTER 5

Management and Considerations

5.1 Ethical, Legal and Security Issues

5.1.1 Ethical issues

A hosted function platform asks a user to place their code and their users' data on a third party's hardware. That is a reasonable trade for many and it is not available to everyone: a hospital, a school holding information about minors and a government department under a residency rule each face the same question, which is whose employees could in principle read this. A platform that requires an account somewhere cannot answer it favourably.

Cubicle's ethical position follows from one decision: nothing in the product contacts anything outside the machine it runs on. There is no registration, no telemetry, no crash reporting, no analytics and no licence check, and the console loads no asset from an external origin. That is not a privacy policy, which is a promise, but a property of the build, which can be verified by watching the network while using the product.

One feature is a deliberate exception and is treated as one. The optional assistant sends source code to a language model provider; it is off unless a key is configured, and the endpoint is configurable, so pointing it at a model inside the same network keeps the property intact. An operator who does nothing gets a product that talks to nobody. Presenting an assistant as a default and mentioning the data flow in small print would be the unethical version of the same feature.

The strongest obligation in a security-adjacent product is not to overstate what it protects. Operating system containers are not a hardware enforced boundary, and describing them as one would put a reader's data at risk on the strength of a marketing sentence, so the limitation is stated in the repository documentation and again here in the same words. The same principle governs metering: a request that failed before reaching the handler is charged zero, because charging an internal team for a platform failure would turn a page that exists to settle arguments into a source of them. A registry that installs somebody else's code is likewise a supply chain, so nothing installs without being asked, nothing updates itself, the complete source is displayed first, and the installer refuses to write secrets on the user's behalf.

5.1.2 Legal issues

Cubicle is released under the Apache License 2.0. A copyleft licence would discourage the institutional adoption the project is aimed at, since operators there frequently modify a self-hosted platform to fit an internal system, while a permissive licence without a patent grant would give less protection than Apache-2.0. Every dependency was checked for compatibility.

Table 5.1: Principal third party components and their licences

Component	Licence	Use
FastAPI, Pydantic, SQLAlchemy, Alembic, asyncpg	MIT and Apache-2.0	HTTP layer, request validation, persistence, migration and the database driver
cryptography, argon2-cffi, docker SDK	Apache-2.0, BSD and MIT	Envelope encryption, password hashing, the scheduling substrate client
React, React Router, CodeMirror, Tailwind, Vite, TypeScript	MIT and Apache-2.0	Console, in-browser editor, styling and build tooling
PostgreSQL, Caddy	PostgreSQL Licence, Apache-2.0	Control plane state; edge and certificate management
Redis 7	BSD-3-Clause	Sessions and the log channel. Version 7 predates the 2024 licence change and the pinned image is used unmodified
Sora, IBM Plex Mono	SIL Open Font Licence	Bundled with the console rather than fetched, which the licence permits

Redis changed licence after version 7; the project pins version 7 and uses the published image unmodified, so no obligation from the later licence attaches. An operator who upgrades that image takes on whatever

the newer terms require, and the documentation says so.

Cubicle is not a data controller and holds no user's data itself; the operator controls whatever their functions process. What the platform can do is make the controller's obligations easier to meet, in four ways relevant to the General Data Protection Regulation and comparable regimes. Residency becomes a property of where the machine is, so there is no cross-border transfer to analyse. Security of processing is addressed by encryption at rest with a key held outside the database, argon2id hashing, TLS by default and confined execution. Erasure is complete rather than approximate, because deleting a cluster cascades through everything it owns. And data is minimised: invocation records hold timing, status and size rather than bodies, and logs are pruned after fourteen days. Two obligations remain with the operator and are documented rather than assumed: keeping the root key safe, and deciding what their own functions log.

5.1.3 Security issues

Figure 5.1 shows the trust boundaries. The administrator password is stored only as an argon2id hash and the plaintext is never written to storage or a log. Sessions are opaque tokens held in Redis rather than self-contained signed tokens, which is the less fashionable choice and the correct one here: a server side session can be revoked, so signing out or changing a password takes effect immediately. API keys carry a recognisable prefix so a leaked key can be identified in a log or a repository scan, only a keyed hash is stored, and a key is displayed exactly once.

Authorisation is enforced in one place: every cluster-scoped endpoint obtains its cluster through a single dependency that performs the access check, so a new endpoint cannot forget it, and both gates are applied, the API key's scope first and then the account's grants.

Each secret gets its own AES-256-GCM data key, wrapped by a key derived through HKDF from a root key that lives in the environment file and never in the database. Sealed material is bound to its record, so a ciphertext copied elsewhere will not decrypt, and values reach a container in the invocation payload rather than being written to its filesystem, so a stolen backup yields nothing. The corresponding cost is equally deliberate: if the root key is lost every stored secret becomes undecryptable, and the console says so in place of the value.

Isolates run with the strongest containment the mechanism allows: an unprivileged user, a read only root filesystem, every capability dropped, `no-new-privileges` set, a process limit, memory and swap capped to the same value so a limit cannot be evaded by swapping, and a small temporary filesystem as the only writable path. Request bodies are bounded at 6 MB before parsing, source bundles at 2 MB with every file name checked, and README content, which may arrive from the marketplace, is rendered into React elements rather than injected as HTML.

Two things are worth saying without hedging. **Containers are not a hard multi-tenant boundary**: isolation comes from the host kernel, and a kernel vulnerability is a path between tenants. The mechanisms that give a hardware enforced boundary are not available on an unmodified container host, so where genuinely untrusted code must run the documented guidance is to put those clusters on separate hosts. **The control plane holds the container socket**, and a process that can reach it can create containers on the host, which is equivalent to root there. That is the price of being the thing that starts isolates; the mitigation is architectural, and the documentation states that access to the control plane should be treated as root on the machine.

5.2 Limitations and Challenges

Four problems took materially more effort than expected.

Scale-down did not happen. The first reclamation policy used only an idle timeout, and pools that grew during a burst never shrank. The cause was an interaction between two correct decisions: acquisition takes the least used idle container in order to spread load, and spreading load keeps every timestamp fresh, so nothing was ever idle for long enough. The fix was the second, shorter mechanism described in Section 3.1.4.

Binary payloads were destroyed in transit. An image uploaded to a function arrived as text, because the request path decoded the body as UTF-8 with replacement and the response path had the mirror problem. The repair was to carry non-text bodies as base64 with an explicit flag in both directions. The lesson recorded here is about diagnosis rather than encoding: the failure looked identical before and after the first fix, because

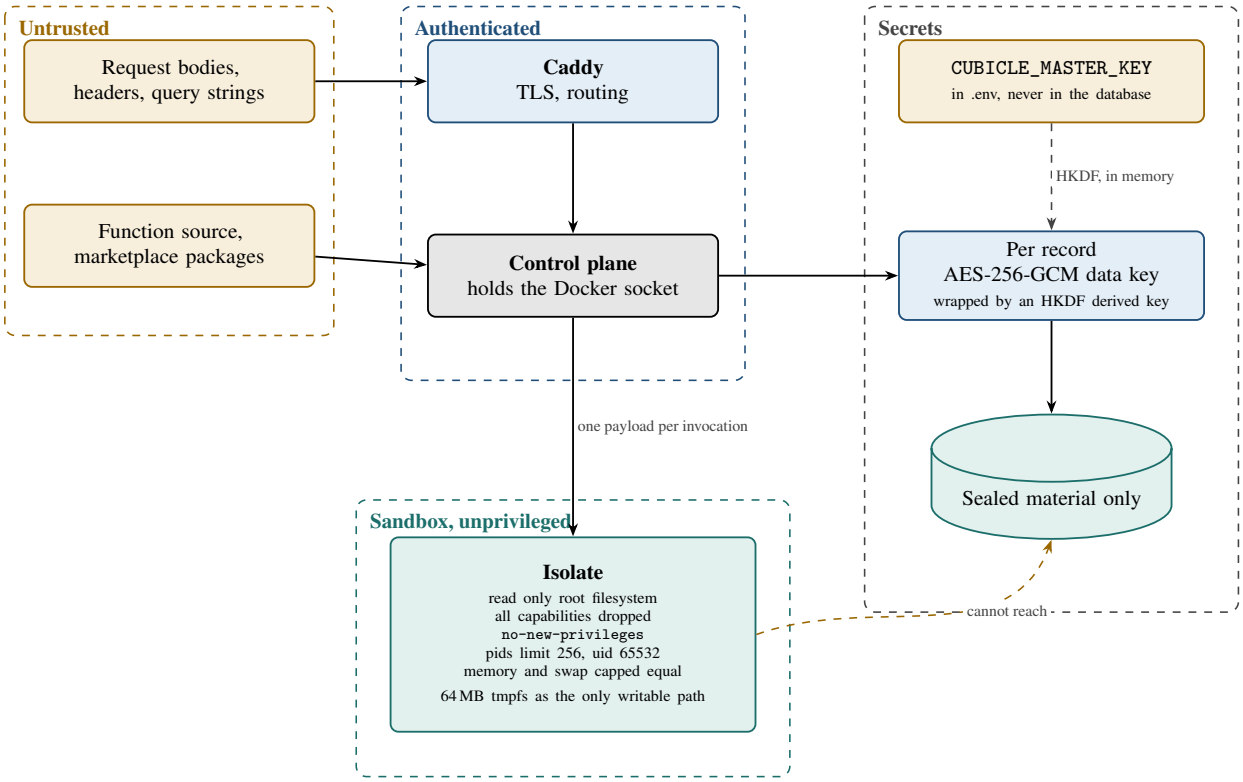


Figure 5.1: Trust boundaries and the envelope encryption path. The root key never enters the database; the database never holds anything but sealed material.

Table 5.2: Known limitations

Limitation	Description	Route to removing it
Single control plane	It can be restarted without losing warm containers, but while it is down no new invocation is routed.	State is already entirely in PostgreSQL and Redis and trigger claiming is already safe for concurrent processes, so running several behind the edge is a deployment change.
Isolation is not hardware enforced	Discussed in Section 5.1.3.	A micro virtual machine backend behind the same pool interface, on hosts that support it.
Placement is least loaded, not bin packed; no automated backup	The scheduler does not consider affinity, spread or fragmentation, and the operator is told to back up the environment file and database volume rather than the product doing it.	A pluggable placement policy, and a scheduled dump that the trigger machinery could already drive.
Marketplace is read only	Browsing and installing are implemented; publishing produces a document to submit rather than a call to a service.	A publishing endpoint, deferred so the consuming half could be finished first.
Runtime rebuild is manual	An agent change requires an explicit rebuild.	Stamp the agent version into the image and compare it on load, so the console can show that a rebuild is needed.
Cold start floor	A container create and start is a few hundred milliseconds and cannot be made much smaller with this mechanism.	Warm instances above zero on latency critical functions, which is implemented.

the edge had started sending base64 while the agent had not yet learned to decode it. An identical symptom is not evidence of an unchanged cause.

An update that only half applied. That defect is also the clearest example of a structural problem, described in Section 3.1.4: the agent lives inside the runtime image, the image tag never moved during development, and the stale image was reused silently.

A body model that silently became a query parameter. A schema class used in a route signature was never imported, and because the module used deferred annotation evaluation the name resolved to an unresolvable string rather than raising, so the framework quietly demoted what should have been a request body. The fault surfaced only when the OpenAPI document failed to generate, and generating that document is now part of the check, because it exercises every route signature at once.

One threat to validity should be stated: the performance figures were taken on developer machines under Docker Desktop, not on server hardware under sustained load. They are adequate for the claims made from them, which concern orders of magnitude and relative behaviour, and they are not a benchmark.

5.3 Team Collaboration and Time Management

5.3.1 Division of work

The project was carried out by a group of five. Work was divided by architectural layer rather than by feature, so each member owned an interface and could work without waiting on the others, and integration happened at those interfaces.

Table 5.3: Division of work within the group

Member	Primary responsibility	Principal contribution
Tamzidul Haque	Control plane, data model, scheduler	Domain model and migrations, the isolate pool and invocation path, cluster boundary and access control, resource ceilings and the scheduler
Puja Chowdhury	Console and user interface	Console shell and routing, playground and editor, dashboard and metering views, mobile layout, bundled documentation
Umma Iman Monea	Runtime images, SDK, command line client	Runtime images and the in-container agent, the runtime registry, the bundled SDK, the command line client, the marketplace client
Tamima Rahman	Testing, security review, documentation	Test suite, the reconciler, security review of the containment and encryption paths, and the written documentation
Shagato Chowdhury	Architecture, deployment, release	Architecture, the installer and edge configuration, integration across the layers, and the release
Shared by the whole group		
Requirements analysis, architecture, code review, this report		

5.3.2 Practices and time management

Two practices kept five people working on one system from treading on each other. Every non-obvious claim was measured rather than asserted: the memory floor, the concurrency behaviour, the reclamation timeline, the even spread of work, the exactly once property of trigger claiming and the byte for byte round trip of a binary payload were each established by running the system, and every number in this report came from that process. And faults were reproduced before they were fixed, so the reconciler was validated by manufacturing real drift rather than by a test double that would have agreed with the implementation's assumptions.

Sixty-three commits were made across the eight working days, through a single shared branch with review before merge. Two conventions were held to. A commit message states what the change achieves rather than what was edited, so the history reads as a record of decisions. And every commit leaves the system installable, which is what makes the incremental model of Section 2.1 true rather than aspirational.

Work was organised as increments with a deployable exit criterion rather than as timed blocks, because a fixed period spent on a task produces an unfinished task while a defined exit criterion produces a result. Three rules governed the ordering. The riskiest work came first, so the warm pool was built in the first increment: if the model had not worked, the rest of the project would have needed a different design. Blocked

work was set down rather than pushed, which is why marketplace publishing appears in Table 5.2 as a known limitation rather than a half-finished feature. And verification was not deferred to the end, which is why the test suite covers the parts where a silent defect would be most expensive.

Acknowledgement

We are grateful to Dr. Abdus Sattar, Associate Professor and Director, M.Sc. in CSE, Department of Computer Science and Engineering, for supervising this work. The guidance that mattered most to us was not on how to write the code but on what a report of this kind has to establish, and on the difference between describing a system and defending its design.

We thank the faculty of the Department of Computer Science and Engineering at Daffodil International University for the grounding in software engineering, operating systems, databases and computer networks that this project draws on. The parts of Cubicle that were hardest to get right, namely process isolation, concurrency control and transactional guarantees, are coursework applied to a real system rather than anything invented for it.

Within the group we would like to record our thanks to each other. Dividing a system of this size by architectural layer only works if everyone honours the interface they own, and each member did so consistently: the pool never had to know how the console rendered, and the console never had to know how a container was started. The daily review of one another's work caught more defects than any single tool did, and the willingness to have a design decision argued with, and sometimes reversed, is the reason several of the mechanisms described in Chapter 3 are simpler than the versions first proposed.

This project stands on a great deal of free and open source software. The maintainers of FastAPI, SQLAlchemy, PostgreSQL, Redis, React, CodeMirror, Caddy and Docker have given away work of a quality that a student project could not otherwise reach, and the licence terms in Table 5.1 are the only thing any of them asked in return.

Finally we thank our classmates for the discussions that repeatedly turned out to be more useful than the answers they were looking for, and our families for their patience and support while this was being built. Any errors that remain in this report are ours.

Tamzidul Haque
Puja Chowdhury
Umma Iman Monea
Tamima Rahman
Shagato Chowdhury

9 August 2026

References

[1] Docker Inc., "Docker Engine API Reference." [Online]. Available: https://docs.docker.com/reference/api/engine/

[2] S. Ramírez, "FastAPI Documentation." [Online]. Available: https://fastapi.tiangolo.com/

[3] M. Bayer, "SQLAlchemy 2.0 Documentation." [Online]. Available: https://docs.sqlalchemy.org/

[4] Meta Open Source, "React Documentation." [Online]. Available: https://react.dev/

[5] M. Holt and the Caddy authors, "Caddy Documentation: Automatic HTTPS." [Online]. Available: https://caddyserver.com/docs/automatic-https

[6] Amazon Web Services, "AWS Lambda Developer Guide." [Online]. Available: https://docs.aws.amazon.com/lambda/

[7] A. Ellis and the OpenFaaS community, "OpenFaaS Documentation." [Online]. Available: https://docs.openfaas.com/

[8] The Apache Software Foundation, "Apache OpenWhisk Documentation." [Online]. Available: https://openwhisk.apache.org/documentation.html

[9] I. Baldini et al., "Serverless Computing: Current Trends and Open Problems," in *Research Advances in Cloud Computing*, Springer, 2017, pp. 1-20.

[10] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981.